ACI UNIVERSAL PAYMENTS

DATABASE OPERATIONS MANUAL

ACI PROACTIVE RISK MANAGER™
RELEASE 9.0.2.0

ACI WORLDWIDE, INC.

Table of Contents

Document Control

The following table lists the changes that have been made to the *Database Operations Manual*.

DATE	RELEASE	SECTION	DESCRIPTION OF CHANGE
25-Feb-2019	8.8.3.0	9.4.1	SQL Server behavior during rule procedure redeploy
21-Nov-2018	8.8.2.0	4.2.2	Modified the description of indexes for partitioned EP tables.
		12.2	Modified the description of indexes for partitioned EP tables
		12.2.2.2	Removed this topic “Index hint on calculation merge.”
07-Dec-2018	8.8.2.0	12.2	EP performance overview
19-Sept-2019	9.0.0.0	5.2	Non Enhanced Profile Tables - added DAYSTOKEEP details for POC_FRAUDDETAIL table
19-Sept-2019	9.0.0.0	8.3	Mentioned the new password encryption approach for DB passwords.
26-Sept-2019	9.0.0.0	16	Added an appendix for deprecated hardware and software platforms
14-April-2020	9.0.1.0	2.4.3	Added information about the updated column order of the SHORTWINDOW and DETAIL clustered indexes.
14-April-2020	9.0.1.0	3.1	Added PRM jobs PP_DASHB_AP_CALC and PP_DASHB_QST_CALC.
14-April-2020	9.0.1.0	4.16.1	Added information about the A&R Dashboard Rules Performance Report calculation.
14-April-2020	9.0.1.0	4.16.2	Added information about the Dashboard Rule Performance Report cleanup.
14-April-2020	9.0.1.0	4.17.1	Added information about the A&R Dashboard Analyst Performance Report calculation.
14-April-2020	9.0.1.0	4.17.2	Added information about the A&R Dashboard Analyst Performance Report cleanup.

14-April-2020	9.0.1.0	4.18	Added information about the PP_DASHB_QST_CALC procedure.
14-April-2020	9.0.1.0	6.11	Updated Non-partitioned tables – statistics.
25-Jun-2020	9.0.2.0	15	PRM Telemetry
25-Jun-2020	9.0.2.0	15.1	Rules Event Alerting

1 Introduction

This document describes how to run and maintain Proactive Risk Manager (PRM) after initial installation. It includes information on when standard PRM jobs should be run to keep important tables up to date with recent transaction activity.

Information is also provided to help database administrators determine the database maintenance strategy, ensuring that system performance is maintained.

However, even if all the steps in this document are followed, aggregates and rules pose a significant risk to performance. To mitigate this risk there is a rule writing best practice document which contains advice on processes to put in place and guidance for writing efficient rules.

Even if operational staff are not currently involved in the rule writing process, they should review rules before being promoted into production, and be familiar with the rules best practice manual.

2 Partitioning

2.1 Overview

The purpose of partitioning DETAIL and SHORTWINDOW tables is for efficient cleanup operations; data is eliminated at the partition level rather than using DELETE.

In addition, partitioning allows for database maintenance, such as statistics and index rebuild, to be run in a realistic maintenance window.

The purpose of partitioning is not to support parallel processing of SQL queries.

Starting from 9.0 version (for clean install), the PARTITIONING parameter became obsolete. Even if the parameter is switched to 0, it will be forced to default to 1 (partitioned).

This parameter remains for backward compatibility (available only for PRM upgrades from 8.2 until 8.8.3 version).

The reason is that PRM supports only partitioned database starting with PRM 8.8 version (Enhanced Profile refactoring by using only partitioned tables).

2.2 Installation options

Partitioning is optional and is controlled by the DB.PARTITIONING installation configuration parameter:

0 = NOT use partitioning (Default)

1 = ALLOW to use partitioning for tables and indexes

In general, this parameter should be set to 1, to allow partitioning. The only reason not to enable partitioning is if the database is not licensed for partitioning. This may be true for development or test environments; however, for production environments it is strongly recommended to use enterprise edition of the database, with partition option licensed in Oracle.

SHORTWINDOW, RT_ SHORTWINDOW and DETAIL tables are the only partitioned tables created at installation time. The RULE_TESTER_DATASET table is also partitioned when created.

Restriction:

Starting with PRM 8.8, with the existing Enhanced Profile feature redesign (which use exclusively the partitioning), you need the Oracle Partitioning feature license; otherwise, PRM install or upgrade can’t be performed. MsSql Enterprise Edition already contains the Partitioning feature.

2.3 Partition size

By default, the SHORTWINDOW table has daily partitions and DETAIL has weekly partitions. In most cases there should be no reason to change the interval of SHORTWINDOW table. However, depending on transaction volume and retention period it may be desirable to increase the number of days per partition of the DETAIL table.

From a storage efficiency perspective, it may be desirable to have daily partitions in detail. This would mean that no data older than the retention period would be stored. In addition, database maintenance

tasks such as index rebuild would only need to be run against 1 day of data, so take less time. Further Insert performance would be better as smaller indexes cost less to maintain.

However, queries whose data is spread over 100 days will have to touch 100 partitions, this may cause slow performance in the UI.

Going to the other extreme, to optimize UI performance there would be a very small number of partitions. Perhaps for a 90-day retention period 3 partitions of 30 days. However, in a high-volume environment this is not practical as DB maintenance tasks (index rebuild/stats/backup) would take too long and Insert performance may not meet NFR. In addition, at the end of the 30-day period there would be 120 days of data retained rather than 90, which may cause space problems.

Thus, the optimum number of days per DETAIL table partition is a balance between UI performance, Insert performance and DB maintenance. The larger the transaction volume the smaller number of days each partition can contain.

The default of 7 days is a good balance between UI performance and DB maintenance tasks, and should suit most shops. However, the number of days per partition can be changed if required.

2.4 Partitioning Implementation

2.4.1 Oracle

Oracle's interval partitioning is used, with this type of partitioning is that Oracle automatically creates partitions when rows are inserted that don't fit into any existing partitions. This has many benefits, however, there are some drawbacks as well.

The main advantage is that without any intervention rows are in the correct partition. So, when processing enters a new day there is no need to create a new partition in advance.

However, when a new partition is created in this way the statistics are empty, thus if a query runs only against the new partition the optimizer assumes the partition is empty, when in fact it contains data. One way to address this issue is to gather stats in the current day partition a short time before midnight, and then copy the statistics to the next day's partition

This is discussed in more detail in the **STATISTICS** section.

Another drawback of interval partitioning is if 'old' transactions enter the system. For example, if there is a retention period of 3 days in the SHORTWINDOW table and 5 transaction which are 5,6,7,8 ad 9 days old. There is no catch all partition, so each of these transactions will create a new partition. This may cause a performance issue with rules which don't contain the DD_PARTITION_DATE safety net.

To avoid this happening these 'old' transactions should be filtered before arriving in PRM.

These 'old' partitions will get cleaned up when PP_PARTITION_DECISION is run, but until then may cause performance issues with queries that don't contain a predicate on DD_PARTITION_DATE.

A similar issue exists for any future

By default, the interval for detail is 7 days and for short window in 1 day. These intervals can be altered changing the following parameters:

v_day_per_partition

v_number_partitions

in:

..\Model_Upgrade\Model\ORACLE\Objects\Tables\NRT\ create_shortwindow_detail_tables_NRT.sql

2.4.2 MSSQL

Partitioning is achieved using a separate partition schema and function for each partitioned table. By default, partitioning is configured as:

DETAIL - 17 partitions - 7 days per partition

SHORTWINDOW - 7 partitions - 1 day per partition

The start date for both is the current date, for example, the date of installation. Thus, for DETAIL the day of the week a new partition is populated depends on the day of installation. This is important from a database maintenance perspective as it may be that the install happens on a Tuesday, however, the partition index rebuild window is on a Sunday night, so there will be many days the partition is in a fragmented state.

All default behavior can be changed in:

..\Model_Upgrade\Model\MSSQL\Objects\Tables\NRT\create_Partition_schema_function.sql

To make any changes to this file NRT.zip must be unzipped, changes made and a new NRT.zip created.

If the retention period is changed in this file column CLEANUP_TABLE.DAYSTOKEEP also needs to be amended for the corresponding table.

It is important that new partitions are created before data enters the system for those partitions. If this is not done MSSQL will insert transactions into the top most partition. When the new partition is created MSSQL will then move all the transactions to the new partition. Depending on the number of transactions this could take a significant amount of time.

PP_PARTITION_DECISION deletes old partitions and creates new partitions, so this job needs to be run before data enters the system and must be run daily.

2.4.3 Index column position with new order

Starting with 9.0.1 at the PRM fresh install, the SHORTWINDOW and DETAIL clustered indexes are changed by having different column order for performance reason. The previous column order (DD_PARTITION_DATE, SD_TIEBREAKER, ND_KEY_HASH, RECORD_SOURCE_ID) was generating lots of page split because the first index column was constantly the same (DD_PARTITION_DATE). By moving the SD_TIEBREAKER (which is hashed) on the first index position because its random values will decrease the page split. For the existing database there is a way to change the column order in the clustered indexes by stopping the PRM and running the scripts from the “Additional scripts” folder (under 9000_9010 folder), which drop the existing index and recreate with the new position.

3 PRM Batch Jobs Overview

3.1 PRM jobs

Each of these jobs are discussed in detail further in this document. However, the following table gives an overview of how frequency each job should be run.

All jobs supplied with the PRM system are listed here, however, some of them are defined as optional.

PROCEDURE	FREQUENCY
PP_ACCOUNTTIMEOUT	Every 10 minutes
PP_CLEANUP_DECISION	Nightly
PP_PARTITION_DECISION	Nightly
PP_COPY_DETAIL_TO_POC_FRAUDDETAIL and PP_GENPOCALERTS	Frequency is dependent upon business needs. PP_COPY_DETAIL_TO_POC_FRAUDDETAIL should be run prior to PP_GENPOCALERTS and, therefore, it is recommended that these procedures be contained in the same job.
PP_ACTIVE_AL_REPORT	Nightly
*PP_CARD_PROFILES_JOB	Nightly
*PP_MER_PROFILES_JOB	Nightly
*PP_UAN_PROFILES_JOB	Nightly
*PP_TERM_PROFILES_JOB	Nightly
*PP_EMPLOYEE_PROFILES_JOB	Nightly
*PP_UAID_PROFILES_JOB	Nightly
*PP_UAN_PROFILES_JOB	Nightly
*PP_ACCT_PROFILES_JOB	Nightly
*PP_IP_PROFILES_JOB	Nightly
*PP_DEVICE_PROFILES_JOB	Nightly
*PP_PAYEE_PROFILES_JOB	Nightly
PP_ENH_PROFILE_CALC	Every 10 minutes if enhanced profiles are being used
PP_ENH_PROFILE_CLEANUP	Nightly
PP_TCR_STATISTIC	Nightly
PP_DASHB_RULE_PERF_CALC	Frequency is dependent upon business needs. Applicable only for users with A&R interface
PP_DASHB_AP_CALC	Frequency is dependent upon business needs. Applicable only for users with A&R interface
PP_DASHB_QST_CALC	Frequency is dependent upon business needs. Applicable only for users with A&R interface

Create dataset job	Ad Hoc - Defined as a database job but is only run when required

Note: On MSSQL databases, set the job step to execute PP_CLEANUP_DECISION stored procedure without an explicit transaction. Do not wrap the execution of PP_CLEANUP_DECISION stored procedure with a begin transaction/commit transaction/rollback transaction block as batch delete will be affected.

4 PRM Batch Jobs - Detail

4.1 Standard Profile Jobs

Important: Starting with 9.0.2 Release, the Profile Jobs can be scheduled from the UI, using several configuration options. For more details please check the Application Administrator User Guide.

If any jobs were previously scheduled from the database, they must be stopped and rescheduled from the UI. The application administrator user has access by default to all configuration and management screens for Profile Jobs Management, however any user can be granted permission.

Eleven profile jobs are available which are similar to one another, with the main difference being the field that drives the profile calculations. All profile jobs have a daily profile table; however, there are differences in the number of days over which other profiles are calculated. The table below describes which tables are populated by each job. At the end of the job, the relevant rules are run against the newly updated tables.

JOB NAME	STORED PROCEDURES CALLED	TABLE IMPACTED
PP_ACCT_PROFILES_JOB	PP_ACCT_DAILY_PRFL	ACCT_DAILY_PROFILE
	PP_ACCT_3_DAYS_PRFL	ACCT_3_DAYS_PROFILE
	PP_ACCT_31_DAYS_PRFL	ACCT_31_DAYS_PROFILE
PP_CARD_PROFILES_JOB	PP_CARD_DAILY_PRFL	CARD_DAILY_PROFILE
	PP_CARD_DAILY_PEER_PRFL	CARD_DAILY_PEER_PROFILE
	PP_CARD_3_DAYS_PRFL	CARD_3_DAYS_PROFILE
	PP_CARD_3_DAYS_PEER_PRFL	CARD_3_DAYS_PEER_PROFILE
	PP_CARD_31_DAYS_PRFL	CARD_31_DAYS_PROFILE
	PP_CARD_31_DAYS_PEER_PRFL	CARD_31_DAYS_PEER_PROFILE
PP_DEVICE_PROFILES_JOB	PP_DEVICE_DAILY_PROFILE	DEVICE_DAILY_PROFILE
	PP_DEVICE_3_DAYS_PROFILE	DEVICE_3_DAYS_PRFL
	PP_DEVICE_7_DAYS_PROFILE	DEVICE_7_DAYS_PRFL
PP_EMPLOYEE_PROFILES_JOB	PP_EMPLOYEE_DAILY_PRFL	EMPLOYEE_DAILY_PROFILE
	PP_EMPLOYEE_DAILY_PEER_PRFL	EMPLOYEE_DAILY_PEER_PROFILE
	PP_EMPLOYEE_7_DAYS_PRFL	EMPLOYEE_7_DAYS_PROFILE
	PP_EMPLOYEE_7_DAYS_PEER_PRFL	EMPLOYEE_7_DAYS_PEER_PROFILE
	PP_EMPLOYEE_30_DAYS_PRFL	EMPLOYEE_30_DAYS_PROFILE
	PP_EMPLOYEE_30_DAYS_PEER_PRFL	EMPLOYEE_30_DAYS_PEER_PROFILE
	PP_EMPLOYEE_90_DAYS_PRFL	EMPLOYEE_90_DAYS_PROFILE
	PP_EMPLOYEE_90_DAYS_PEER_PRFL	EMPLOYEE_90_DAYS_PEER_PROFILE
PP_IP_PROFILES_JOB	PP_IP_DAILY_PROFILE	IP_DAILY_PROFILE
	PP_IP_3_DAYS_PROFILE	IP_3_DAYS_PROFILE

JOB NAME	STORED PROCEDURES CALLED	TABLE IMPACTED
	PP_IP_7_DAYS_PROFILE	IP_7_DAYS_PROFILE
PP_MER_PROFILES_JOB	PP_MER_DAILY_PRFL	MER_DAILY_PROFILE
	PP_MER_DAILY_PEER_PRFL	MER_DAILY_PEER_PROFILE
	PP_MER_7_DAYS_PRFL	MER_7_DAYS_PROFILE
	PP_MER_7_DAYS_PEER_PRFL	MER_7_DAYS_PEER_PROFILE
	PP_MER_30_DAYS_PRFL	MER_30_DAYS_PROFILE
	PP_MER_30_DAYS_PEER_PRFL	MER_30_DAYS_PEER_PROFILE
	PP_MER_90_DAYS_PRFL	MER_90_DAYS_PROFILE
	PP_MER_90_DAYS_PEER_PRFL	MER_90_DAYS_PEER_PROFILE
PP_PAYEE_PROFILES_JOB	PP_PAYEE_DAILY_PROFILE	PAYEE_DAILY_PROFILE
	PP_PAYEE_3_DAYS_PROFILE	PAYEE_3_DAYS_PROFILE
	PP_PAYEE_7_DAYS_PROFILE	PAYEE_7_DAYS_PROFILE
PP_TERM_PROFILES_JOB	PP_TERM_DAILY_PRFL	TERM_DAILY_PROFILE
	PP_TERM_DAILY_PEER_PRFL	TERM_DAILY_PEER_PROFILE
	PP_TERM_3_DAYS_PRFL	TERM_3_DAYS_PROFILE
	PP_TERM_3_DAYS_PEER_PRFL	TERM_3_DAYS_PEER_PROFILE
	PP_TERM_7_DAYS_PRFL	TERM_7_DAYS_PROFILE
	PP_TERM_7_DAYS_PEER_PRFL	TERM_7_DAYS_PEER_PROFILE
PP_UAID_PROFILES_JOB	PP_UAID_DAILY_PRFL	UAID_DAILY_PROFILE
	PP_UAID_3_DAYS_PRFL	UAID_3_DAYS_PROFILE
	PP_UAID_31_DAYS_PRFL	UAID_31_DAYS_PROFILE
PP_UAN_PROFILES_JOB	PP_UAN_DAILY_PRFL	UAN_DAILY_PROFILE
	PP_UAN_DAILY_PEER_PRFL	UAN_DAILY_PEER_PROFILE
	PP_UAN_3_DAYS_PRFL	UAN_3_DAYS_PROFILE
	PP_UAN_3_DAYS_PEER_PRFL	UAN_3_DAYS_PEER_PROFILE
	PP_UAN_31_DAYS_PRFL	UAN_31_DAYS_PROFILE
	PP_UAN_31_DAYS_PEER_PRFL	UAN_31_DAYS_PEER_PROFILE

If the profiles are required then the relevant job should be run nightly, after midnight, to calculate the profiles for the previous day. Profile jobs can be run in parallel as they insert into different tables.

If all of the profiles for a particular job are not required, the JOB stored procedure can be edited. The stored procedures listed in the above table can be commented out. However, all the multi day profiles (for example, 3,7,30 days) rely on the daily profile data. Thus, the daily profile must always be run.

Old profile data is deleted in the PP_CLEANUP_DECISION procedure based on retention periods in CLEANUP_TABLES. The minimum retention period for the daily table is the largest number of days in a multiday profile for that profile column. If daily profiles are deleted too early, then the multi day profiles will not be correct. For example, if PP_EMPLOYEE_90_DAYS_PRFL is run then the minimum retention period for EMPLOYEE_DAILY_PROFILE is 90 days.

4.2 Enhanced Profiles

4.2.1 Enhanced profile management tables

4.2.1.1 ENH_PROFILE_SETUP table

Column PROFILE_FREQUENCY_ID (number), contains the frequency interval for the profile.

Column LAST_CALCULATION_DATE (datetime), contains the last date when the calculation was run for the profile.

Column FREQUENCY_MOVE_DELAY_DATE (datetime), contains the date and time when the calculation will run after changing the frequency for the profile.

4.2.1.2 ENH_PROFILE_FREQUENCY table

Column PROFILE_FREQUENCY_ID (numeric), contains the frequency interval for the profile.

Column PROFILE_FREQUENCY_NAME (varchar), contains the label name of the frequency interval.

Column PROFILE_FREQUENCY_MINUTES (number), contains the number of minutes of the frequency interval.

Column PARALLEL_ENABLED (number), contains the flag that enables/disables the parallel calculation mode.

Column LAST_FILL_ENH_TEMP_DATE (datetime), contains the flag that enables the calculation procedure to rebuild (based on the new enhanced profile changed).

4.2.1.3 ENH_PROFILE_NEW_TXNS table

Column DD_SYSDATE (datetime), contains the current datetime of the inserted row (autofill).

This table is partitioned after this column, by using the lowest frequency interval partition.

4.2.1.4 ENH_TEMP table

Column DD_SYSDATE (datetime), contains the current datetime of the inserted row (autofill).

This table is partitioned after this column, by using the lowest frequency interval partition.

4.2.2 Enhanced profile indexes

The non-partitioned EP tables have 3 indexes: first index based on primary_field, secondary_field, and cycle_end_date columns, second index based on secondary_field, primary_field, and cycle_end_date columns, and third index based on cycle_end_date (used for cleanup).

The partitioned EP tables have one index based on primary_field, secondary_field and cycle_end_date columns and one index based on secondary_field, primary_field and cycle_end_date columns (only if secondary field is defined). These indexes are partitioned.

4.2.3 Enhanced profile stored procedures

There are 6 jobs which support enhanced profiles:

- 5 jobs for PP_ENH_PROFILE_CALC - updates profiles with most recent transactions.
- 1 job for PP_ENH_PROFILE_CLEANUP - delete old profile data based on cycles to keep.

4.2.3.1 PP_ENH_PROFILE_CALC

This is a dynamically created stored procedure, it is created when:

- a new Enhanced Profile is defined
- an Enhanced profile is edited
- an Enhanced profile is deleted

The system provides five pre-defined intervals that can be changed, providing flexibility to use intervals starting from 10 minutes to 24 hours, and to switch an enhanced profile between groups without any restrictions or additional operations. The enhanced profile calculation procedure PP_ENH_PROFILE_CALC is called with the argument to specify the frequency interval id.

The lowest frequency interval is set by default to 60 minutes, and the highest to 24 hours.

The user can assign the enhanced profiles to the lowest frequency the one that needs to be up to date, and to highest the other one, to avoid performance impact by distributing the calculation task on different time slots.

Note that enhanced profiles are calculated in series rather than in parallel. This is to minimize the impact on the system, especially during peak processing.

Calculation is done via a single MERGE statement for each profile, this either INSERTs new data or UPDATEs existing data, incrementing any counters/totals.

If the DBMS scheduler is being used and the job runs for more than 60 minutes then the next job will not start until the currently executing job is complete. Once the currently executing job completes then next job will start immediately and not wait for the next 60-minute slot. The duration of the enhanced profile job depends on:

Number of enhanced profiles defined

Number of tables joined in the profile

Volume of transactions being processed

If many complex enhanced profiles are defined it is possible that the job may run for more than 10 minutes during peak processing.

When defining an enhanced profile there is an option to add a filter to the transactions used to populate the profile. From a business perspective this is very useful, however, it should also be considered from a performance perspective. The smaller the number of candidate transactions the quicker the enhanced profile calculation will be.

Please see the Enhanced Profile manual for more information on this.

All five enhanced profile jobs are calling the main stored procedure named PP_ENH_PROFILE_CALC using the first argument to specify the frequency group ID for which is running. The second argument is to debug the messages in SystemAudit table.

Example of calculation procedure call from scheduler job:

- Oracle: execute immediate 'begin PP_ENH_PROFILE_CALC(v_PROFILE_FREQUENCY_ID => :v_frequency_id); end;' using 1;
- MsSql: exec PP_ENH_PROFILE_CALC @PROFILE_FREQUENCY_ID=1

For the new PRM install, the EP scheduler jobs are automatically created and there is no need to perform manual configuration, because can be done directly from UI.

For the existing PRM upgrade, the recommendation is to remove the existing EP scheduler job for calculation and cleanup, because these jobs are automatically created.

The main stored procedure named PP_ENH_PROFILE_CALC (based on the value sent on the frequency ID argument), calls the stored procedure named PP_ENH_PROFILE_CALC_<frequency_ID>, like for example: PP_ENH_PROFILE_CALC_1.

When the main PP_ENH_PROFILE_CALC procedure is called, first checks if the column CHANGED from ENH_PROFILE_FRQUENCY table has value greater than 0 (which means that was changed, like for example, by adding a new enhance profile), rebuild the PP_ENH_PROFILE_CALC_<frequency_ID> procedure and run it. If the CHANGED value is 0 the procedure isn't rebuilt and just run it.

The Frequency Groups enhancement allows to select the interval at which a calculation procedure is run to update an enhanced profile's data. The predefined intervals are:

For frequency ID = 1, 24 hours; ID = 2, 12 hours, ID = 3, 6 hours, ID = 4, 3 hours, ID=5, 1 hour.

The ENH_PROFILE_NEW_TXNS table collects (only the unique transaction identifier columns) the last transactions (since last calculation) thru PP_ENH_PROFILE_NEW_TXNS procedure.

The ENH_TEMP table is the replication of unique transaction identifier columns from ENH_PROFILE_NEW_TXNS joined with DETAIL table only on primary and/or secondary fields defined by enhanced profiles. This data replication (done by PP_ENH_PROFILE_FILL_ENH_TEMP procedure) is called (by default) only in the lowest frequency interval (ID=5), which mean in PP_ENH_PROFILE_CALC_5 procedure. The lowest frequency job will handle first the data replication in ENH_TEMP, and secondly will continue with the calculation for the enhanced profiles from this group, by updating the LAST_FILL_ENH_TEMP_DATE column from ENH_PROFILE_FREQUENCY table with the current datetime. Based on this update, the other scheduled jobs will include the last replication in their calculation. So, if this job (with ID=5) is stopped, the other jobs will continue to perform their calculation, by merging 0 rows, since ENH_TEMP table doesn't have new data. For example, if this job is started after

two days of delay, it will replicate data in ENH_TEMP table, and all jobs will take time to calculate the new two days of data.

If the enhanced profiles calculation from the lowest frequency group takes longer than the predefined interval time, it's recommended to move them in the next low group, to not affect the ENH_TEMP data replication, which could delay the other calculation jobs from the data availability perspective.

When a new/change/delete enhanced profiles action takes place, it's possible to see the next stored procedures: PP_ENH_PROFILE_CALC_99 and PP_ENH_PROFILE_FILL_ENH_TEMP99 (ends with ID=99), because are dummy procedures generated just for syntax check and raise the error message in UI. They are generated and maintained by the PP_ENH_PROFILE_ALTER_ALL procedure. The original calculation procedure can't be rebuilt from UI, because it will cause the lock and SAF-ing.

4.2.3.2 PP_ENH_PROFILE_CLEANUP

This job should be run every night. It is recommended that the calculation job is paused whilst the cleanup job is running to ensure there are no lock contention issues between the two jobs. The Enhanced profile cleanup job deletes rows from the Enhanced profile tables based on the cycle length and retention period for each enhanced profile.

The upgraded system will continue to run using both types of enhanced profiles: existing (non-partitioned) and newly (partitioned).

Like partitioned tables such as DETAIL, cleanups are handled by dropping aged partitions instead of removing aged rows (one by one), greatly improving clean-up performance.

It is recommended that retention periods for enhanced profiles are kept to the minimum required, to keep the tables as compact as possible. For example, if rules against an enhanced profile reference data up to 3 days old, there is no point in retaining data for any more time than that.

The ENH_PROFILE_NEW_TXNS and ENH_TEMP table cleanup is done automatically by the replication PP_ENH_PROFILE_FILL_ENH_TEMP procedure with transparent messages in SystemAudit table.

The cleanup from PP_ENH_PROFILE_FILL_ENH_TEMP and PP_ENH_PROFILE_CLENUP procedures automatically detects the aged partitions and add the future partitions, even if case of previous procedure failure.

On MsSql, the new enhanced profile is created with the same number of retention in the past and future.

For example, a daily enhanced profile with 10 days retention period, will have automatically the 10 partition days in the past (according with the retention period) and 10 partition days in the future (for safety).

On both PP_ENH_PROFILE_CALC and PP_ENH_PROFILE_CLEANUP jobs are handled and controlled by using Task semaphore enhancement.

Task_semaphore table seed for enhanced profile:

- EP_PROFILE_SETUP -Enhanced profile setup used to create/modify/remove an EP.

- EP_PROC_FILL_ENH_TEMP - Enhanced profile task used by the PP_ENH_PROFILE_FILL_ENH_TEMP procedure to fill ENH_TEMP with DETAIL data.
- EP_PROC_CALC_RUN_FOR_FREQ_1 -Enhanced profile task used to signal the already running calculation procedure for frequency group 1.
- EP_PROC_CALC_RUN_FOR_FREQ_2 -Enhanced profile task used to signal the already running calculation procedure for frequency group 2.
- EP_PROC_CALC_RUN_FOR_FREQ_3 -Enhanced profile task used to signal the already running calculation procedure for frequency group 3.
- EP_PROC_CALC_RUN_FOR_FREQ_4 -Enhanced profile task used to signal the already running calculation procedure for frequency group 4.
- EP_PROC_CALC_RUN_FOR_FREQ_5 -Enhanced profile task used to signal the already running calculation procedure for frequency group 5.
- EP_FREQUENCY_CHANGE -Enhanced profile task used to signal the already running frequency changes.

TASK_SEMAPHORE table has RUN_FLAG column used to signal the current job status, by using 0 when it's not running, and 1 when it currently is running. For example, when PP_ENH_PROFILE_CALC_2 is running, its RUN_FLAG is set to 1, so it can't be started again at the same time by raise a specific error message. In case of extreme failure, the flag can be reset by using the specific task semaphore procedure (stop), but usually the errors are handle inside the procedure and the task semaphore is reset automatically.

4.2.3.3 PP_ENH_PROFILE_CALC additional script

A new additional script (can be run on demand, will not be automatically installed) was created starting with PRM 8.6.3 for existing Enhanced profiles when the data type needs to be changed from Money to Numeric (32,2). The script applies only to MSSQL Server databases and can be used to avoid an arithmetic overflow error.

File PP_ENH_ALTER_MONEY_TYPE_msq.sql

- It will change the Money data type to NUMERIC(32,2) for existing enhanced profiles depending on their storage: rows < 5 mil, and rows > 5 mil
- The Stored Procedure contains the following parameters:

@error VARCHAR(255) OUTPUT
@enh_profile_name VARCHAR(100) - the enhanced profile name as stated in setup table
@enh_profile_id INT - the enhanced profile id as stated in setup table
@BATCH INT = 500000 - default batch value, can be changed by calling the SP with this parameter

The Stored Procedure uses the ENH Semaphore. No other ENH process should be able to run until this Stored Procedure releases the semaphore.

- For enhanced profiles with rows < 5mil:

The Stored Procedure will change directly the data type for TXNS_AMOUNT column except TXNS_MIN and TXNS_MAX Amount with an ALTER command.

- For enhanced profiles with rows > 5mil:

The Stored Procedure will create another new table with the exact structure as the original enhanced profile table.
It will:
add the primary key as the original table has,
copy the data in batches,
add indexes,
rename the tables (the original table will contain _OLD_MONEY suffix), then rename all indexes to both original and new table to be compliant with EHP naming.

- Example for calling the Stored Procedure: with the default batch:

```
DECLARE @ERROR VARCHAR(MAX)
EXEC PP_ENH_ALTER_MONEY_TYPE @ERROR = @error
OUTPUT, @enh_profile_name = 'YourENHProfileName',
@enh_profile_id = 1
```

- Example for calling the Stored Procedure with another batch value:

```
DECLARE @ERROR VARCHAR(MAX)
EXEC PP_ENH_ALTER_MONEY_TYPE @ERROR = @error
OUTPUT, @enh_profile_name = 'YourENHProfileName', @enh_profile_id = 1, @BATCH = 400000
```

4.3 PP_ACCOUNTTIMEOUT

This procedure cleans expired sessions, deleting any account locks timeout sessions may have. It is run by the UI server when a user logs off or on, however, it is possible for a significant period to pass without any logon/logoff activity. Therefore, it is recommended this job to be scheduled to run every 10-15 minutes, it could be scheduled as a step in the enhanced profiles job.

4.4 PP_TCR_STATISTIC

4.4.1 TCR_GROUP table is storing the transaction group.

COLUMN_NAME	DATA_TYPE	MAX_LENGTH	IS_NULLABLE
GROUP_ID	int	N/A	NO
GROUP_NAME	nvarchar	50	NO
SQL_CRITERIA	nvarchar	MAX	NO
GROUP_ORDER	int	N/A	NO
INCLUDED	varchar	1	NO
ACTIVE	varchar	1	NO
CREATE_DATETIME	datetime	N/A	NO

COLUMN_NAME	DATA_TYPE	MAX_LENGTH	IS_NULLABLE
MODIFY_DATETIME	datetime	N/A	YES
SHORTWINDOW_TABLE	varchar	30	NO
DETAIL_TABLE	varchar	30	NO
AMOUNT_FIELD_NAME	varchar	30	NO
AMOUNT_ABSOLUT_VALUE	varchar	1	NO
GROUP_CKSUM	int		YES
ACTIVE_DATETIME	datetime		YES

Column description:

GROUP_ID - This column contains the unique group ID.

GROUP_NAME - This column contains the non-unique group name.

SQL_CRITERIA - This column contains the criteria used to filter transaction tables Shortwindow/Detail.

GROUP_ORDER - This column contains the order of processing the count.

INCLUDED - This column contains the flag 'Y' for Included group or 'N' for Excluded group.

ACTIVE - This column contains the flag 'Y' for Active group or 'N' for Inactive group.

CREATE_DATETIME - This column contains the date time when the group was created.

MODIFY_DATETIME - This column contains the date time when the group was changed.

SHORTWINDOW_TABLE - This column contains the Action Level Shortwindow table predefined for current group.

DETAIL_TABLE - This column contains the Action Level Detail table predefined for current group.

AMOUNT_FIELD_NAME - This column contains the amount filed on which will be used to build SUM statistics.

AMOUNT_ABSOLUT_VALUE - This column contains the flag 'Y' to compute negative amounts, 'N' to compute only positive amount values.

GROUP_CKSUM - This column contains the checksum value performed on the values defined on the current group.

ACTIVE_DATETIME - This column contains the last datetime when the group active flag was changed.

4.4.2 TCR_STATISTIC table

This table is the repository of statistics (Count & Sum) collected based on active transaction groups.

COLUMN_NAME	DATA_TYPE	MAX_LENGTH	IS_NULLABLE
STATISTIC_DATE	Date	N/A	NO
GROUP_ID	Int	N/A	NO
STATISTIC_COUNT	Int	N/A	NO
STATISTIC_AMOUNT	Money	N/A	NO
STATISTIC_CKSUM	Int	N/A	YES
LAST_RUN_DATE	Date	N/A	YES
DETAIL_TABLE	Varchar	30	NO

Column description:

STATISTIC_DATE - This column contains the date based on which statistical data is grouped.

GROUP_ID - This column contains the transaction group ID.

STATISTIC_COUNT - This column contains the total number of transaction found for this group.

STATISTIC_AMOUNT - This column contains the total cumulative amount of transaction found for this group, excluding the negative values.

STATISTIC_CKSUM - This column contains the checksum value performed on the values defined on the current group.

LAST_RUN_DATE - This column contains the date when the statistical data was done. It will be used in order to exclude adding multiple times the same statistical data to the existing one.

DETAIL_TABLE - This column contains the Action Level Detail table predefined for current group.

4.4.3 TCR_RUN_LOG

This table is used to keep the log of collecting process

COLUMN_NAME	DATA_TYPE	MAX_LENGTH	IS_NULLABLE
RUN_DATE	date	N/A	NO
GROUP_ID	int	N/A	NO
DETAIL_TABLE	varchar	30	NO
TRANSACTION_TABLE	varchar	30	NO
RUN_STATE	varchar	1	NO
RUN_MESSAGE	varchar	400	YES
LOG_DATETIME	datetime	N/A	NO

Column description:

RUN_DATE - This column contains the date used to collect statistics, same with LAST_RUN_DATE column from TCR_STATISTIC table

GROUP_ID - This column contains group ID reference with TCR_GROUP

DETAIL_TABLE - This column contains the Action Level Detail table predefined for current group.

TRANSACTION_TABLE - This column contains the name of table used to collect statistic data (Detail or Shortwindow).

RUN_STATE - This column contains the flag 'Y' for statistics collected successfully or 'N' for statistics collected for current date (or in the future) or 'E' for other errors.

RUN_MESSAGE - This column contains the message of successful action, or the error message if it was running against current day, or the error message if the action crashed.

LOG_DATETIME - This column contains the date and time of the current action.

4.5 Functionality

4.5.1 Adding new transaction group

The simplest way to call PP_TCR_GROUP_ADD procedure is using at least 2 parameters: Group name, Sql criteria. By default, if the Active and Included flag is not specified, it will be setup default to TRUE ('Y'), SHORTWINDOW, DETAIL table and MD_TRAN_AMT1 column is used by default, AMOUNT_ABSOLUT_VALUE flag setup default to 'Y' (True). Also the Debug flag is setup TRUE by default. So,

To get the Warning or Info messages only on Oracle, before any action, enable the environment output with the following command:

```
exec dbms_output.enable
```

The parameters value can be specified using the parameter name.

Values like 61 from SD_TRAN_CDE_CAT must be included in double quote.

MSSQL:

```
exec PP_TCR_GROUP_ADD 'credit card transaction', 'SD_TRAN_CDE_CAT=''61'' AND
MD_TRAN_AMT1>100'
```

Oracle:

```
exec dbms_output.enable

exec PP_TCR_GROUP_ADD ('credit card transaction', 'SD_TRAN_CDE_CAT=''61'' AND
MD_TRAN_AMT1>100')
```

OR

MSSQL:

```
exec PP_TCR_GROUP_ADD @GROUP_NAME='credit card transaction',
@SQL_CRITERIA='SD_TRAN_CDE_CAT=''61'' AND MD_TRAN_AMT1>100'
```

Oracle:

```
exec dbms_output.enable

exec PP_TCR_GROUP_ADD (v_GROUP_NAME=>'credit card transaction',
v_SQL_CRITERIA=>'SD_TRAN_CDE_CAT=''61'' AND MD_TRAN_AMT1>100')
```

If you need to specify ALL procedure parameters:

MSSQL:

```
exec PP_TCR_GROUP_ADD @GROUP_NAME='credit card transaction',
@SQL_CRITERIA='SD_TRAN_CDE_CAT=''61'' AND MD_TRAN_AMT1>100', @GROUP_ORDER=9,
@INCLUDED='Y', @ACTIVE='Y', @SHORTWINDOW_TABLE='SHORTWINDOW',
@DETAIL_TABLE='DETAIL', @AMOUNT_FIELD_NAME='MD_TRAN_AMT1', @bDebug=1
```

Oracle:

```
exec dbms_output.enable

exec PP_TCR_GROUP_ADD (v_GROUP_NAME=>'credit card transaction',
v_SQL_CRITERIA=>'SD_TRAN_CDE_CAT=''61'' AND MD_TRAN_AMT1>100',
v_GROUP_ORDER=>9, v_INCLUDED=>'Y', v_ACTIVE=>'Y',
v_SHORTWINDOW_TABLE=>'SHORTWINDOW', v_DETAIL_TABLE=>'DETAIL',
v_AMOUNT_FIELD_NAME=>'MD_TRAN_AMT1', v_bDebug=>1)
```

The MOST usual procedure call is:

MSSQL:

```
exec PP_TCR_GROUP_ADD @GROUP_NAME='credit card transaction',
@SQL_CRITERIA='SD_TRAN_CDE_CAT=''61'' AND MD_TRAN_AMT1>100', @GROUP_ORDER=9,
@INCLUDED='N', @ACTIVE='Y'
```

Oracle:

```
exec dbms_output.enable

exec PP_TCR_GROUP_ADD (v_GROUP_NAME=>'credit card transaction',
v_SQL_CRITERIA=>'SD_TRAN_CDE_CAT=''61'' AND MD_TRAN_AMT1>100',
v_GROUP_ORDER=>9, v_INCLUDED=>'N', v_ACTIVE=>'Y')
```

4.5.2 Modify an existing transaction group

The procedure can be called specifing at least the group ID.

The usual procedure call is for changing the sql criteria:

MSSQL:

```
exec PP_TCR_GROUP_MODIFY 7, @SQL_CRITERIA='MD_TRAN_AMT1>11001'
```

Oracle:

```
exec dbms_output.enable

exec PP_TCR_GROUP_MODIFY (7, v_SQL_CRITERIA=>'MD_TRAN_AMT1>11001')
```

If you need to specify ALL procedure parameters:

MSSQL:

```
exec PP_TCR_GROUP_MODIFY 7, @GROUP_NAME='credit card transaction2',
@SQL_CRITERIA='MD_TRAN_AMT1>11001', @GROUP_ORDER=1, @INCLUDED='N',
@ACTIVE='N', @AMOUNT_FIELD_NAME='MD_TRAN_AMT1', @bDebug=1
```

Oracle:

```
exec dbms_output.enable

exec PP_TCR_GROUP_MODIFY (7, v_GROUP_NAME=>'credit card transaction2',
v_SQL_CRITERIA=>'MD_TRAN_AMT1>11001', v_GROUP_ORDER=>1, v_INCLUDED=>'N',
v_ACTIVE=>'N', v_AMOUNT_FIELD_NAME=>'MD_TRAN_AMT1', v_bDebug=>1)
```

4.5.3 Transaction Cont procedure which collect statistics

Specify the date and flag RUN_MANUALLY parameter, which can be 'Y' or 'N' (Automatically mode = 'N', and Manually mode = 'Y') for what date to collect statistics.

Scheduler job will be configured to call `PP_TCR_STATISTIC` procedure with parameter `RUN_MANUALLY setup to 'N'`.

For manual calculation, the user will call `PP_TCR_STATISTIC` procedure with parameter `RUN_MANUALLY setup to 'Y'`.

MSSQL:

```
begin

  SET DATEFORMAT MDY

  DECLARE @DD_DATE DATE

  SET @DD_DATE = CAST('03.26.2013' AS DATE)

  exec PP_TCR_STATISTIC @PROCESSED_DATE=@DD_DATE, @RUN_MANUALLY='N', bDebug=1

end
```

Oracle:

```
exec dbms_output.enable

exec PP_TCR_STATISTIC (v_PROCESSED_DATE=> to_date('03.26.2013 00:00:00',
'mm.dd.yyyy HH24:MI:SS'), v_RUN_MANUALLY=>'N', v_bDebug=>1)
```

4.5.4 Transaction Count LOG

Every time when the PP_TCR_STATISTIC is called for a specific date, in TCR_RUN_LOG table is written a log which mean a row for each transaction group found active and used to gather statistics.

The main purpose is for tracking the customer action, when the date specified to the PP_TCR_STATISTIC procedure is the current day or a day in the future.

When the reports are display, it will show also the dates when the procedure was running for current days, and set flag 'N' in RUN_STATE column.

For more information, RUN_MESSAGE column will contain the details.

Also the procedure errors (when the procedure is failing) will be track, but with a different flag 'E' in RUN_STATE column. For successful actions it will set flag 'Y' in RUN_STATE.

4.6 Display data from database

4.6.1 Transaction group query

Group ID 1 with name "Orphans" is inserted into database at the installation time (seed data). The values used for SHORTWINDOW_TABLE, DETAIL_TABLE colums are one space character, because the columns are Not Null and this group is generic used by all other transaction group defined on diffrent Detail tables.

```
select GROUP_ID, GROUP_NAME, SQL_CRITERIA, GROUP_ORDER, INCLUDED, ACTIVE,
ACTIVE_DATETIME, CREATE_DATETIME, MODIFY_DATETIME,

SHORTWINDOW_TABLE, DETAIL_TABLE, AMOUNT_FIELD_NAME, AMOUNT_ABSOLUT_VALUE

from TCR_GROUP
```

4.6.2 Transaction statistics data query

The statistics gather for one specific date is grouped at 3 columns: `STATISTIC_DATE, GROUP_ID, DETAIL_TABLE.`

```
select STATISTIC_DATE, GROUP_ID, DETAIL_TABLE,

STATISTIC_COUNT, STATISTIC_AMOUNT, LAST_RUN_DATE

from TCR_STATISTIC
```

4.6.3 Transaction log query

```
TCR_RUN_LOG table is used to log all the collecting process which is called
every day.
```

The log table for one specific date is grouped at 3 columns: `RUN_DATE, GROUP_ID, DETAIL_TABLE.`

```
select RUN_DATE, GROUP_ID, DETAIL_TABLE, TRANSACTION_TABLE,

RUN_STATE, RUN_MESSAGE, LOG_DATETIME

from TCR_RUN_LOG
```

4.7 Logging actions

Every procedure contain one parameter named Debug which is default setup to 1 (which mean TRUE), and it's used to track the actions in SYSTEMAUDIT table.

It can be setup to 0 (which mean FALSE), and the log tracking is disable.

For example, when `PP_TCR_GROUP_ADD` procedure will insert the message `'Add the new transaction group ID: 3'` into SystemAudit table using type 11 and subtype

The ' Transaction Count' subtype (100) is defined under 'Script' type (11) in SYSTEMAUDITSUBTYPES table.

4.8 Rule tester dataset creation

This is defined as a database job during installation, however, it is run on an ad hoc basis when a new dataset is created. It should not be run as a scheduled job.

This job copies a potentially large amount of data from the DETAIL table to the dataset table, during which time machine resources will be consumed. If this is run during peak processing it is possible that transaction processing performance may be impacted.

By default the create dataset job is enabled but not scheduled, when a user requests a new dataset is created from the UI after some initial set up tasks PP_START_CREATE_DATASET_JOB is called which starts the job:

MSSQL - msdb.dbo.sp_start_job
Oracle - DBMS_SCHEDULER.run_job

4.9 Total Fraud Losses report job

Total Fraud Losses report needs to get from transaction table the most recent date.

Because of performance reason, instead of querying DETAIL table, a new job needs to be configured in order to search the recent date from SHORTIWINDOW table, and store them into a new table named TFL_REPORT_LAST_DD_DATE.

Important!

This report will use data (from this table) updated by the new nightly job, described below.

TFL_REPORT_LAST_DD_DATE table description:

- *ACTION_LEVEL_ID* column - INTEGER - primary key, keeps the action level id (from 1 to 7)
- *ACTION_LEVEL_KEY* column - STRING - primary key, keeps the action level master key (SM_UAN).
- *DD_DATE DATE* column - DATE (without time) - primary key, keeps the value of the most recent transaction from SHORTWINDOW (default).

- *LAST_ACTIVE_DAYS* column - INT (integer) - value of no. of days from orig. mark date of the most recent transaction that occurred after the original mark date

PP_TFL_REPORT procedure functionality:

- collects the number of days from the original mark date to the date of the most recent transaction that occurred after the original mark date as an integer value, and the DD_DATE of last transaction occurred.
- cleans up any orphaned action level keys and rebuild the clustered index from TFL_REPORT_LAST_DD_DATE table, on the first day of each month.

PP_TFL_REPORT procedure parameters:

- bShortwindow – flag with possible values: 1 (true, used by default) and !=1 (false), used to switch the search on SHORTWINDOW (default) or on DETAIL table
- bDebug - flag with possible values: 1 (true, used by default) and !=1 (false), to write in SystemAudit table the log message of success or failed operation.

Job usage:

For example, a new MSSQL job can be created and configured to run every night, before cleanup job, with the following command:

exec PP_TFL_REPORT

or using the following parameters:

exec PP_TFL_REPORT @bShortwindow = 1, @bDebug = 1

For example, after migration from 7.1 to 8.3, the command can be played manually to gather data, by searching for last transaction date from DETAIL, instead of SHORTWINDOW, by following:

exec PP_TFL_REPORT @bShortwindow = 0, @bDebug = 1

Warning!

If you delete a marked UAN's record from UAN_MASTER table, the Avg Active Days statistic and the statistics involving Funds at Risk won't be accurate, since TFL_REPORT_LAST_DD_DATE.LAST_ACTIVE_DAYS and UAN_MASTER.MM_FUNDSATRISK won't be available. You can avoid this by deleting only UANs from UAN_MASTER if they have never been marked.

4.10 PP_TRENDS_CALCULATION_JOB Procedure

PP_TRENDS_CALCULATION_JOB procedure gather statistical data for Trends Tab chart from UI, for each action level specified as procedure parameter.

You can schedule your own nightly job to run for the active action levels, by default is gathering statistical data from previous day (yesterday).

4.10.1 Trends calculation

Trends calculation job will be schedule to run every night (to gather statistical data for Trends Tab chart) named PP_TRENDS_CALCULATION_JOB.

PP_TRENDS_CALCULATION_JOB store procedure has 4 (IN) parameters:

v_start_date parameter (if is not specified default value is set to yesterday), used to define the start interval used for calculation,
v_end_date parameter (if is not specified default value is set to yesterday), used to define the end interval used for calculation,
v_action_level_id parameter (if is not specified default value is 1, which is Card action level), used to define the action level ID used for calculation,
v_bDebug parameter (if is not specified default value is 1, which is true), used to activate the SystemAudit logging for calculation.

ORACLE:

Example of (usual calculation by) calling the stored procedure from nightly job for CARD action level (id = 1) on last day complete transactions (yesterday):

begin

PP_TRENDS_CALCULATION_JOB(v_action_level_id => 1);

end;

Example of running the stored procedure manually for a defined date interval (last 1 week):

begin

PP_TRENDS_CALCULATION_JOB(v_action_level_id => 1, v_start_date => sysdate-7, v_end_date => sysdate-1);

end;

SQL SERVER:

Example of (usual calculation by) calling the stored procedure from nightly job for CARD action level (id = 1) on last day complete transactions (yesterday):

begin

exec PP_TRENDS_CALCULATION_JOB @action_level_id = 1

end

Example of running the stored procedure manually for a defined date interval (last 1 week):

declare

@start_date date = DATEADD(DD, -7, getdate()),
@end_date date = DATEADD(DD, -1, getdate())
begin
exec PP_TRENDS_CALCULATION_JOB @start_date = @start_date, @end_date = @end_date,

@action_level_id = 1
end

In case of wrong start date, end date, or out of range of end date from DETAIL table retention days period, an error message will be raised and logged in SystemAudit.

Since some action levels have DAILY Trends table defined, and some not, for the ones which have DAILY table the rest of the tables will be updated based on this, and for the other will be updated based on Shortwindow or Detail table depending by the calculation date where is the best (from performance perspective) way to be calculated.

Debug flag can be unset by assigning the 0 value to v_bDebug parameter in the PP_TRENDS_CALCULATION_JOB stored procedure call. If this parameter is not specified, default value 1 is set and enables to log all calculation steps in SytemAudit table (and available in UI under Script -> Trends Graph node from SystemAudit window).

The end date is validated based its end interval period, for example for an AL which starts with weekly cycle (doesn't have daily) the calculation job will first calculate the end cycle date (for the end date parameter specified) and this value will be validated if it's contained in Detail table.

The calculation job procedure validates the parameters and raise an error message in the following conditions:

- The start date exceeds the end date return error message: "Start date <start date value> exceeds end date <end date value>"
- The end date exceeds that data that is available in DETAIL and does not execute the job return error message: "End date <end date value> exceeds available data"
- The start date is set to a date in the future return error message: "Start date <start date value> is in the future"
- When deleting old data when recalculating the statistics for a date range over which they were previously calculated return error message: "Deleted <number of records>records from <table name> table with end date cycle: <end date cycle>"
- When deleting old data when recalculating the statistics for a date range over which they were previously calculated (for partitioned table) return error message: "Deleted records from <table name> table (with DROP PARTITION) with end date cycle <end date cycle>"
- When inserting new data, including the number of records inserted return error message: "Merged <number of records> into <table name> table from <SW/DET/DAILY> table."
- When starting the calculations return error message: "Started <Grouping name> <action level name> calculation with end date cycle <end date cycle>"
- When completing the calculations return error message: "Finished <Grouping name> <action level name> calculation with end date cycle <end date cycle>"

When the job fails to commit return error message: "Failed <Grouping name> <action level_name> calculation. SQLERRM: <database error message>"

4.10.2 Trends historical calculation

When Trends is initially installed, the Trends calculation job can be used to calculate historical data.

When calculating historical data, the Trends calculation should be run month by month, instead of over 3 months/the full retention in Detail (continuous interval), for performance reasons. The start date should =

the 1st of the selected month, and the end date should = the last date of the selected month. The Trends calculation job cannot be used on data that is older than 1 year.

Avoid using, for example, weekly intervals because bi-monthly and monthly intervals which are already calculated will be overwritten (delete/insert operation), impacting the database performance.

Oracle performance was done on following PRM 8.6 standard with the following data distribution:

Table name	Rows	No. partitions	Partition type
SHORTWINDOW	126,000,000	3	Daily
DETAIL	4,200,000,000	14	Weekly
CARD_MASTER	40,000,000		
MERCHANT_MASTER	250,000		
UAN_MASTER	60,000		
TERMINAL_MASTER	250,000		
EMPLOYEE_MASTER	100,000		
UAID_MASTER	20,000,000		
ACCOUNT_MASTER	60,000,000		

Note:
All Trends calculations (90 days - historical data) were running using one single CPU core.
OS: Linux. DB: Oracle 12.1 (pluggable database).
Elapsed time for each AL/Cycle/Rows in minutes:

Action level	Cycle (type)	Rows	Elapsed time (minutes)
Card	Weekly	40,000,000	27
Card	Bimonthly	40,000,000	55
Card	Monthly	40,000,000	110
Merchant	Daily	250,000	8
Merchant	Weekly	250,000	0.2
Merchant	Bimonthly	250,000	0.1
UAN	Weekly	60,000	30
UAN	Bimonthly	60,000	60
UAN	Monthly	60,000	120
Terminal	Daily	250,000	4
Terminal	Weekly	250,000	0.2
Terminal	Bimonthly	250,000	0.1
Employee	Weekly	100,000	50
Employee	Bimonthly	100,000	120

Employee	Monthly	100,000	300
UAID	Weekly	20,000,000	35
UAID	Bimonthly	20,000,000	65
UAID	Monthly	20,000,000	110
Account	Weekly	60,000,000	50
Account	Bimonthly	60,000,000	100
Account	Monthly	60,000,000	180

The elapsed time duration depends by the number of AL KEYs, by the CPU speed, and by disk speed. The HDD used with 10k rpm and 15k rpm, with response time for read around 5-7 ms, and for write around 10-14 ms. This table (with elapsed time) can be used as refence and not as exact data, since for example on SSD the response/elapsed time will be dramatically decreased.

4.11 Trends cleanup

The nightly CLEANUP job, which calls PP_CLEANUPPARTITION procedure for partitioned tables or PP_CLEANUP_NON_PARTITIONED procedure for non-partitioned tables, will remove the cycle days older the number of days to be kept defined in CLEANUP_TABLES for all statistical Trends tables.

Default retention per action level and per cycle is 12 records. For example, for DAILY keep the last 12 days, for WEEKLY keep the last 12 weeks, for BIMONTHLY keep the last 12 bi-months, for MONTHLY keep the last 12 months.

Since the CLEANUP_TABLES table store the total number of days to keep and not the number of cycles, for WEEKLY will keep the last 12 cycles x 7 days (1 week) = 84 days, for BIMONTHLY will keep the last 12 cycles x 15 days (1 bi-month) = 180 days, for MONTHLY will keep the last 12 cycles x 31 days (1 month) = 372 days.

Trends table can be installed (during the 8.6 upgrade) as partitioned or non-partitioned, by enabling from properties file the parameter NRT.DB.PARTITIONING=1, if your database license supports partitioning.

On Sql Server with partitioning enable on Trends, calculation job is doing also the cleanup Trends tables automatically. For Sql Server non-partitioning and Oracle (partitioning and non-partitioning) the cleanup tables is done in old fashion way, with cleanup store procedure job.

4.11.1 Trends calculation – use cases

Generic behavior:

Every time, daily table interval calculation for current day are completely refreshed by delete/insert operation, instead of merge.

Trends calculation procedure perform commit after each interval calculation. For example, it performs commit the calculated rows on TRENDS_1_DAILY, commit the calculated rows on TRENDS_1_WEEKLY, commit the calculated rows on TRENDS_1_BIMONTHLY, commit the SystemAudit messages, and not a single commit at the end of entire job.

Scenario 1, when the scheduler job was not running for last day (yesterday).

If the job was not running at all for yesterday, the Calculation_date column was not updated for that day. In this case the trends calculation will automatically perform a merge operation (insert/update) on existing Trends tables with the new rows for weekly, bimonthly, monthly table intervals.

Scenario 2, when the scheduler job was already running for last day.

If the job was already running once, the Calculation_date column was updated for that day. In this case the trends calculation will automatically perform delete and insert back the computed rows for weekly, bimonthly, monthly table intervals.

For example, if the interval range specified is one day, the weekly, bimonthly, monthly table intervals will be recalculated (delete/insert operations) only for that specific interval (week, bi-month or month) which contain the day specified in the calculation.

Example:

*PP_TRENDS_CALCULATION_JOB(v_action_level_id => 1, v_start_date => to_date('**12/13/2016**','mm/dd/yyyy'), v_end_date => to_date('**12/13/2016**','mm/dd/yyyy'));*

Since 13 December was already computed, but because some batch files were loaded, you want to process the calculation again (by taking into account the new batch transactions), the entire week (11 December – 17 December) will be removed and insert back the new calculation, and the same for the entire bi-moth (01 December – 15 December), or for entire month (01 December – 31 December) if the AL has it assigned.

Scenario 3, when the Trends was just installed and need the historical data populated.

Depending by the Detail table size, we recommend to run Trends calculation month by month, instead of 3 months (continuous interval), because one month calculation could fit in your low traffic window every night. Avoid to use, for example, weekly intervals, because bi-monthly and monthly intervals which are already calculated will be overwritten (delete/insert operation), and impact the database performance.

4.12 AG KPI SLA Report

PP_AG_KPI_SLA_REPORT can be run for any date range in the past. This procedure calculates AG SLAs based on AG KPIs stored in the AG_KPI_LOG table. During calculations, it will store daily rollups for each AG in the AG_KPI_SLA_REPORT_STATISTICS table. The results of this report are stored in AG_KPI_SLA_REPORT_RESULT table.

PP_AG_KPI_SLA_REPORT store procedure has 2 (IN) parameters:

v_start_date parameter used to define the start interval used for calculation,

v_end_date parameter used to define the end interval used for calculation

PP_AG_KPI_SLA_REPORT stores results in the AG_KPI_SLA_REPORT_RESULT table.

ORACLE:

Example of running the stored procedure manually for a defined date interval (last 1 week):

```
begin
PP_AG_KPI_SLA_REPORT (v_start_date => sysdate-7, v_end_date => sysdate-1);
end;
```

SQL SERVER:

Example of running the stored procedure manually for a defined date interval (last 1 week):

```
declare

@start_date date = DATEADD(DD, -7, getdate()),
@end_date date = DATEADD(DD, -1, getdate())
begin
exec PP_AG_KPI_SLA_REPORT @StartDateTime = @start_date, @EndDateTime = @end_date
end
```

ORACLE and SQL SERVER:

Example retrieving results from *AG_KPI_SLA_REPORT_RESULT*

```
Select PROCESS_ID,UP_TIME,PREC_OF_TXN_WITHIN_TARGET, NUMBER_TXN_WITHIN_TARGET,
        NUMBER_TXN_NOT_WITHIN_TARGET ,TOTAL_NUMBER_OF_TXN ,MIN_RESPOSNE_TIME ,
        MAX_RESPOSNE_TIME ,AVG_TPS ,NOTES

from AG_KPI_SLA_REPORT_RESULT ORDER BY Process_ID ASC;
```

4.13 UI KPI SLA Report

PP_UI_KPI_SLA_REPORT can be run for any date range in the past. This procedure calculates UI up time as percent based on UI KPIs stored in the UI_KPI. This procedure will check the UI_KPI table for a given source_id , date range and interval in seconds. Calculating up time percent if there are any logging periods longer then the provided interval.

PP_UI_KPI_SLA_REPORT store procedure has 2 (IN OUT) 2 (IN) and 1 (OUT) parameters:

v_start_date IN OUT parameter used to define the start interval used for calculation,

v_end_date IN OUT parameter used to define the end interval used for calculation,

log_interval IN parameter used to define the interval expected between log statements,

sourceID IN parameter used to define the UI KPI to be used for calculation

up_time OUT parameter used to report up time as a percentage

ORACLE:

Example of running the stored procedure manually for a defined date interval (1st week of April):

```
SET SERVEROUTPUT ON

declare

  v_up_time NUMBER(10,2);

  v_start_date date := to_date('2018-04-01 00:00', 'YYYY-MM-DD hh24:mi') ;

  v_end_date date := to_date('2018-04-07 00:00', 'YYYY-MM-DD hh24:mi');

BEGIN

  PP_UI_KPI_SLA_REPORT(v_start_date,v_end_date,890,'/servlet/GetSystemInfo',v_up_time);

  dbms_output.put_line('UP_TIME: ' || v_up_time || ' Start_Date: ' || v_start_date || ' End_Date: ' || v_end_date);

END;
```

SQL SERVER:

Example of running the stored procedure manually for a defined date interval (1st week of April):

```
declare @Start_date DATETIME = '2018-04-01 00:00';

declare @End_date DATETIME = '2018-04-07 00:00';

declare @UP_TIME INTEGER;

begin;

exec PP_UI_KPI_SLA_REPORT @StartDateTime = @Start_date OUT, @EndDateTime =@End_date
OUT, @log_interval = 60, @sourceID = '/servlet/GetSystemInfo', @UP_TIME = @UP_TIME OUT

select @UP_TIME AS UP_TIME,@Start_date AS START_DATE,@End_date AS END_DATE;

end;

go
```

4.14 Active Action Level Report job

PP_ACTIVE_AL_REPORT_MONTHLY – additional script for Oracle database

PP_ACTIVE_AL_REPORT job description:

On 1st of every month Active Action Level Report can be run Monthly, Quarterly, Semiannually or Annually. On the first day of week, the customer can run a Weekly report. In other days of a month/week the only available option is to run a Daily Report.

PP_ACTIVE_AL_REPORT_MONTHLY:

However, if the job for PP_Active_AL_REPORT will not run daily, the data for Active Action Level Report cannot be calculated. Then PP_ACTIVE_AL_REPORT_MONTHLY should be run on demand in order to calculate data for the whole month as long as data still exists in the Detail table.

PP_ACTIVE_AL_REPORT_MONTHLY should be run on demand for the previous month where a calculation day was skipped from daily PP_ACTIVE_AL_REPORT calculation.

Performance optimization when run against monthly large data volume:

The cursor from PP_ACTIVE_AL_REPORT_MONTHLY is based on a SELECT statement from the DETAIL table and uses BULK COLLECT command with a 500 records limit, which performs inserts or updates in the AL_LAST_AP_DATE table. However, the BULK COLLECT limit should be tested against the customer data distribution to find the BULK COLLECT suitable configuration (the limit can be increased or decreased with 200 at a time).

The MERGE statement from PP_ACTIVE_AL_REPORT was done on SHORTWINDOW table.

PP_ACTIVE_AL_REPORT_MONTHLY performance tests on BULK COLLECT limit:

Example of performance tests on BULK COLLECT limit for PP_ACTIVE_AL_REPORT_MONTHLY:

Connect to your NRT schema, and script PP_ACTIVE_AL_REPORT_MONTHLY stored procedure.

Find the line: v_limit int := 500;

Change the above line from v_limit int := 500; to v_limit int := 1000;

Start performance tests for BULK COLLECT limit. Run on demand
PP_ACTIVE_AL_REPORT_MONTHLY stored procedure.

If the performance is lower with v_limit int := 1000; change back to v_limit int := 500;

For performance tests the recommendation is to first start with v_limit int := 500;

As Oracle stipulates that it cannot guaranty efficiency if the LIMIT is higher than 500, unless performance tests are run.

As a rule, BULK COLLECT increases PGA (Process Global Area) meaning the memory used by application session. When using BULK COLLECT with DML statements (insert, update, delete), the Oracle does not optimize them, instead it will perform the DML statement for each row from the BULK COLLECT loop. This means that on one side you use BULK COLLECT to query the data faster, but you create higher overhead when running the DML inside the loop.

On this solution, there was also a limit used for the BULK COLLECT which would help limit the PGA memory used by the application session. As a general rule, when using the LIMIT for BULK COLLECT, the LIMIT value has to be decided after running performance tests on the actual data volume. Oracle stipulates that it cannot guaranty efficiency if the LIMIT is higher than 500, unless performance tests are run.

PP_ACTIVE_AL_REPORT_MONTHLY can be run on demand as follows:

<NRT_SCHEMA_NAME> is your schema name and should replaced

(if your NRT schema is named PRM86NRT then replace <NRT_SCHEMA_NAME> with PRM86NRT):

V_DATE := ADD_MONTHS(TRUNC(sysdate,'MONTH'),-1); will calculate the first day of the previous month

```
DECLARE

V_DATE DATE;

BEGIN

V_DATE := ADD_MONTHS(TRUNC(sysdate,'MONTH'),-1);

<NRT_SCHEMA_NAME>.PP_ACTIVE_AL_REPORT_MONTHLY ( V_DATE );

COMMIT;

EXCEPTION WHEN OTHERS THEN

ROLLBACK;

END;
```

4.15 PP_APR_CALC Procedure

PP_APR_CALC procedure calculates statistics for the Analyst Performance Report based on the user activity.

4.15.1 Analyst Performance Report calculation

We recommend scheduling a nightly job which calls the PP_APR_CALC procedure which, by default, calculates statistics for the previous day (yesterday).

PP_APR_CALC stored procedure has 3 IN parameters and 1 IN OUT parameter:
- actdate_start parameter is used to define the start interval used for calculation. The default value is set to yesterday, if it is not specified.
- actdate_end parameter is used to define the end interval used for calculation. The default value is set to yesterday, if it is not specified.
- bDebug parameter is used to activate the SystemAudit logging for calculation. The default value is 1, if it is not specified. If the value 0 is set for this parameter, then the logging is deactivated.
- nResult parameter may return the following results: 0=OK, 1=Semaphore locked, 2=interval error, 3=error.

In case of setting wrong start date and/or end date, an error message will be raised and logged in the SystemAudit table.

The PP_APR_CALC procedure uses the APR_CALC task from the TASK_SEMAPHORE table.
If the DBMS scheduler is being used and the job is started, then the next job will not start until the currently executing job is complete. Once the currently executing job completes then next job will start immediately.
If one job runs calling the PP_APR_CALC procedure, no other APR_CALC task from the TASK_SEMAPHORE table should be able to run until the calculation is completed.

ORACLE:

```
set serveroutput on
declare v_ret int;
begin
pp_apr_calc (to_date('20171214','YYYYMMDD'), to_date('20171214','YYYYMMDD'), 1, v_ret);

DBMS_OUTPUT.PUT_LINE('v_result='||to_char(v_ret)||' 0=OK,  1=Semaphore locked, 2=interval error,
3=error');
end;
/
```

SQL SERVER:

```
declare @nResult INTEGER;
begin;
exec pp_apr_calc '20171214', '20171214', 1, @nResult = @nResult OUT
select @nResult, ' 0=OK, 1=Semaphore locked, 2=interval error, 3=error';
end;
go
```

The calculation job procedure validates the parameters and displays message in the SystemAudit table, in the following conditions:

- When the start date exceeds the end date, it returns the message: ”Start date <start date value> exceeds end date <end date value>”
- When the start date is set to a date in the future, it returns the message: ”Start date <start date value> is in the future”
- When starting the PP_APR_CALC procedure, it returns the message: “Compute statistics for APR_STATISTIC started for interval <start date value> - <end date value> “
- When successfully completing the PP_APR_CALC procedure, it returns the message “Compute statistics for APR_STATISTIC executed for interval <start date value> - <end date value> “
- When the PP_APR_CALC procedure fails to commit, it returns the error message: ”APR_STATISTIC error = <database error message>”
- When the Task Code ‘APR_CALC’ from the Task_Semaphore table is set to 1, meaning that the Semaphore is locked, the PP_APR_CALC procedure returns the message: ”Task APR_CALC on table TASK_SEMAPHORE is locked. Probably the Calculation job or another APR calculation has started.”

4.15.2 Analyst Performance Report cleanup

The nightly CLEANUP job calls PP_CLEANUPPARTITION procedure for partitioned tables or PP_CLEANUP_NON_PARTITIONED procedure for non-partitioned tables.

There are 2 non-partitioned tables for the Analyst Performance Report defined in the CLEANUP_TABLES table.
The APR_MARK_HISTORY table contains the Mark and Unmark actions performed by the users.
The default retention period for the APR_MARK_HISTORY table is 90 days.
The APR_STATISTIC table contains the statistics calculated for the Analyst Performance Report.
The default retention period for the APR_ STATISTIC table is 365 days.

4.16 PP_DASHB_RULE_PERF_CALC Procedure

PP_DASHB_RULE_PERF_CALC procedure aggregates data used for the A&R dashboard Rules Performance Report (NRT&RT). Reports based on transactions and alerts.

4.16.1 A&R Dashboard Rules Performance Report calculation

We recommend scheduling a job on customer's needs. It will call the PP_DASHB_RULES_PERF_CALC procedure which, by default, aggregates data for interval between previous calculation and current time.
In case that is first execution, it will compute data for interval between days_to_keep from cleanup table and DB current time. In case of missing information from cleanup table, it will aggregate data for last 90 days

PP_DASHB_RULE_PERF_CALC stored procedure has 1 IN parameters :
- bDebug parameter is used to activate the SystemAudit logging for calculation. The default value is 1, if it is not specified. If the value 0 is set for this parameter, then the logging is deactivated.

4.16.2 Dashboard Rules Performance Report cleanup

The nightly CLEANUP job calls PP_CLEANUPPARTITION procedure for partitioned tables or PP_CLEANUP_NON_PARTITIONED procedure for non-partitioned tables.

There are 3 non-partitioned tables for the Rules Performance Report defined in the CLEANUP_TABLES table.
The TITANIUM_RULE_PERF table contains aggregated data for NRT reports. The default retention period is 90 days.
The TITANIUM_RT_RULE_PERFtable contains aggregated data for RT reports. The default retention period is 90 days.
The TITANIUM_PORT_TOTAL table contains Portfolio aggregated data for NRT reports. The default retention period is 90 days.

4.17 PP_DASHB_AP_CALC Procedure

PP_DASHB_AP_CALC procedure aggregates data used for the A&R dashboard Analyst Performance Report. Reports based on transactions and alerts.

4.17.1 A&R Dashboard Analyst Performance Report calculation

We recommend scheduling a job on customer's needs. It will call the PP_DASHB_AP_CALC procedure which, by default, aggregates data for interval between previous calculation and current time.
In case that is first execution, for DASHB_AP_SYSAUD and DASHB_AP_ACT tables, it will compute all data for interval between days_to_keep from cleanup table for and DB current time

PP_DASHB_RULE_PERF_CALC stored procedure has 1 IN parameters :
- bDebug parameter is used to activate the SystemAudit logging for calculation. The default value is 1, if it is not specified. If the value 0 is set for this parameter, then the logging is deactivated.

4.17.2 Dashboard Analyst Performance Report cleanup

The nightly CLEANUP job calls PP_CLEANUPPARTITION procedure for partitioned tables or PP_CLEANUP_NON_PARTITIONED procedure for non-partitioned tables.

There are 2 non-partitioned tables for the Analyst Performance Report defined in the CLEANUP_TABLES table.
The DASHB_AP_SYSAUD table contains aggregated data for audited actions. The default retention period is 365 days.
The DASHB_AP_ACT table contains aggregated data for Opened account actions. The default retention period is 365 days.

4.18 PP_DASHB_QST_CALC Procedure

PP_DASHB_QST_CALC procedure aggregates data used for the A&R dashboard Queue Status Report. Reports based on transactions and alerts.

4.18.1 A&R dashboard Queue Status Report calculation

We recommend scheduling a job on customer's needs. It will call the PP_DASHB_QST_CALC procedure which, by default, aggregates data for report.

PP_DASHB_QST_CALC stored procedure has 1 IN parameters :
- bDebug parameter is used to activate the SystemAudit logging for calculation. The default value is 1, if it is not specified. If the value 0 is set for this parameter, then the logging is deactivated.

5 Table Access Profiles

5.1 Table volatility

The following table describes the access profile of tables in the PRM database. This information can be used by DBAs as a starting point to develop the maintenance strategy.

Category	Access profile	When	Tables
Transaction tables	INSERT, Truncate Partition	Transaction path Partition cleanup	SHORTWINDOW, DETAIL
Demographic tables	MERGE DELETE - If no recent activity	Demographic load Potentially in transaction path	%_MASTER
Alerts	Inserts and Delete	Frequently	ALERTS, DETAIL_ALERTED_RULE, POCALERTS
UI Actions	Insert Delete during cleanup	Frequently	ACT, FORWARD, LOCKS, SYSTEMAUDIT, ACTION_LEVEL_MEMO
Profiles	Insert during batch job Delete during cleanup	Nightly	%PEER_PROF %PROFILE
Enhanced profiles	MERGE DELETE	Every 10 Minutes Nightly	EP_%
Reference data for rules	INSERT	Ad Hoc	PRM_REF_DATA

5.1.1 Enhanced profile tables

The volatility of each enhanced profile table depends very much on the definition of the table. Factors which impact volatility include:

- Primary field
- Secondary field
- Cycle length
- Filter

For example, an enhanced profile with a daily cycle length and primary field of SD_PAN is likely to have many new records inserted every day, and many deleted during the cleanup process. However, a perpetual enhanced profile, with the same primary field will mostly have updates run against it.

Therefore it is important to understand the definition of each enhanced profile table in order to understand the most appropriate database maintenance approach.

5.2 Data Cleanup Criteria

5.2.1 Non Enhanced Profile Tables

Cleanup of all tables, apart from the enhanced profile tables, is driven by entries in CLEANUP_TABLES. Both partitioned and non-partitioned tables are included in this table.

The table below includes the list of tables and cleanup criteria delivered by default. It is recommended that data is retained only if necessary. For example if only 2 days of Shortwindow data is required by the rules, then only 2 days should be retained.

Table Name	Default DAYSTOKEEP
ACCOUNT_MASTER	365
ACCT_3_DAYS_PROFILE	7
ACCT_31_DAYS_PROFILE	7
ACCT_DAILY_PROFILE	32
ACT	365
ACTION_LEVEL_MEMO	0
ALERTS	30
CARD_3_DAYS_PEER_PROFILE	7
CARD_3_DAYS_PROFILE	7
CARD_31_DAYS_PEER_PROFILE	7
CARD_31_DAYS_PROFILE	7
CARD_DAILY_PEER_PROFILE	32
CARD_DAILY_PROFILE	32
CARD_MASTER	365
DETAIL	90
DETAIL_ALERTED_RULE	90
DEVICE_3_DAYS_PROFILE	7
DEVICE_7_DAYS_PROFILE	7
DEVICE_DAILY_PROFILE	8
EMPLOYEE_30_DAYS_PEER_PROFILE	7
EMPLOYEE_30_DAYS_PROFILE	7
EMPLOYEE_7_DAYS_PEER_PROFILE	7
EMPLOYEE_7_DAYS_PROFILE	7

Table Name	Default DAYSTOKEEP
EMPLOYEE_90_DAYS_PEER_PROFILE	7
EMPLOYEE_90_DAYS_PROFILE	7
EMPLOYEE_DAILY_PEER_PROFILE	91
EMPLOYEE_DAILY_PROFILE	91
EMPLOYEE_MASTER	365
EXTERNAL_MESSAGE_DATA	90
FORWARD	365
IP_3_DAYS_PROFILE	7
IP_7_DAYS_PROFILE	7
IP_DAILY_PROFILE	8
LOCKS	365
MER_30_DAYS_PEER_PROFILE	7
MER_30_DAYS_PROFILE	7
MER_7_DAYS_PEER_PROFILE	7
MER_7_DAYS_PROFILE	7
MER_90_DAYS_PEER_PROFILE	7
MER_90_DAYS_PROFILE	7
MER_DAILY_PEER_PROFILE	91
MER_DAILY_PROFILE	91
MERCHANT_MASTER	365
PAYEE_3_DAYS_PROFILE	7
PAYEE_7_DAYS_PROFILE	7
PAYEE_DAILY_PROFILE	8
POCALERTS	365
PRM_BATCH_HISTORY	90
POC_FRAUDDETAIL	90
POC_HISTORY	90
RULEPERFMONITOR_LOG	3
SHORTWINDOW	3
SYSTEMAUDIT	365
TCR_STATISTIC	365
TERM_3_DAYS_PEER_PROFILE	7
TERM_3_DAYS_PROFILE	7
TERM_7_DAYS_PEER_PROFILE	7
TERM_7_DAYS_PROFILE	7
TERM_DAILY_PEER_PROFILE	8
TERM_DAILY_PROFILE	31
TERMINAL_MASTER	365
TRAVEL_NOTIFICATIONS	365
UAID_3_DAYS_PROFILE	7
UAID_31_DAYS_PROFILE	7
UAID_DAILY_PROFILE	31
UAID_MASTER	365
UAN_3_DAYS_PEER_PROFILE	7
UAN_3_DAYS_PROFILE	7
UAN_31_DAYS_PEER_PROFILE	7
UAN_31_DAYS_PROFILE	7
UAN_DAILY_PEER_PROFILE	32
UAN_DAILY_PROFILE	32
UAN_MASTER	365
TITANIUM_RULE_PERF	90
TITANIUM_RT_RULE_PERF	90
TITANIUM_PORT_TOTAL	90
DASHB_AP_ACT	365
DASHB_AP_SYSAUD	365

5.2.2 Enhanced Profile Tables

Cleanup of each enhanced profile table is determined by the cycle length and cycles retained options of the enhanced profile definition. This information is stored in ENH_PROFILE_SETUP and is used by PP_ENH_PROFILE_CLEANUP.

6 DB Maintenance Schedule

6.1 Non-partitioned tables

This category of tables can be seen to be of a low volatility, with the exception of ALERTS, which is subject to frequent INSERT and DELETE operations.

6.1.1 Non-partitioned tables - statistics

Under normal circumstances collecting statistics on these tables on a weekly basis should be sufficient. However, if there is any significant data change, for example a new set of master data loaded, statistics should be gathered after that event.

By default statistics are gathered during index rebuild operations, so there is no need to collect statistics after index rebuild operations.

Although it is recommended that the auto statistics feature is not used, it may be useful on some low volume but highly volatile tables, such as ALERTS. If it is discovered that poor access paths are being chosen, even with regular statistics, auto statistics is an option that should be considered.

If query response times are slow or unpredictable, ensure that queries have up-to-date statistics before performing additional troubleshooting steps.

You need to determine whether the statistics accurately enough represent the distribution of the data, and update them if it doesn't. The only way to know whether you need to do these things is to track the behavior of queries within your system in order to spot when the optimizer starts making bad choices. This is probably because the statistics are not reflecting the data in a way that leads to good execution plans. You'll need to determine that period based on your system and your circumstances. There's not a formula we can provide that will precisely tell you when to run a manual update on your statistics. Further, we can't tell you how to sample your statistics either. You may be experiencing random sampling that is perfect , or you may need some more specified degree of sampling right up to a full scan. You'll need to experiment to understand what the right answer is in your circumstances.

Oracle allows you to create custom statistics all the way down to creating your own histogram. SQL Server doesn't give you that much control. However, you can create something in SQL Server that doesn't exist in Oracle; filtered statistics. These are extremely useful when dealing with partitioned data or data that is wildly skewed due to wide ranging data or lots of nulls.

Dealing with statistics is one of the more frustrating tasks of maintaining your instances: However, you have many options that enables you to get your statistics optimized on your server. Just remember that just as your data is not always generic, there is no generic solution for the maintenance of statistics. You will need to conform the solution you use to the data and the queries that are running on your system.

6.1.2 Non-partitioned tables - index rebuilds

Index rebuilds should be run if, based on analysis of statistics, it is determined that performance will degrade due to index fragmentation.

However, if there has been a significant change in data then index rebuilds should be considered.

6.2 Partitioned tables

As both Shortwindow and detail are partitioned by date in many environments no data will be added to 'old' partitions. So DB maintenance tasks, like index rebuilds and statistics, only need to be run against current partition or the most recent full partition just after the new partition becomes active.

In some environments however, some data may come in late, due to various operational and/or business reasons. Thus the DBA must have an understanding of how much late data will enter the system, and if it is enough to warrant additional maintenance to be run against old partitions.

6.2.1 Partitioned tables - statistics Oracle

For most objects in the RT and NRT databases statistics should be gathered after the cleanup process has successfully completed. However, if there is any significant load of data, such as new reference data or demographic data load, statistics should be gathered on the effected table. Another exception are the partitioned objects DETAIL and SHORTWINDOW statistics, which need to be approached differently.

When a new partition is created Oracle assumes it is empty, and with all auto stats set to FALSE the optimizer will continue to assume it's empty until stats are gathered. This is an issue as for rule and UI performance as statistics in Shortwindow and Detail must be up to date.

A suggested approach to resolve this issue for Shortwindow table is:

1. Take statistics toward the end of the day (e.g., 11:00 p.m.)
2. Issue a lock partition statement for tomorrow's partition (this creates the partition)
3. Copy the statistics from today's partition into tomorrow's partition.

For the DETAIL a similar approach should be implemented; however these steps should be followed at the end of the 7-day period (or whatever the partition interval is) rather than every day.

Below is a sample script to do the lock partition in step 2, note *schema* will need to be changed:

```
alter session set NLS_TIMESTAMP_FORMAT = "YYYYMMDD
HH24:MI:SS";        declare
v_date date;
begin
select  CURRENT_DATE+1 into v_date from dual;
dbms_output.put_line ('Lock table schema.ShortWindow partition for
(TO_DATE(''' ||v_date ||''','YYYYMMDD HH24:MI:SS')) IN SHARE MODE');

execute immediate 'Lock table schema.ShortWindow partition for (TO_DATE('''
||v_date ||''','YYYYMMDD HH24:MI:SS')) IN SHARE MODE';

commit;
end;
```

Below is a sample statement for step 3. The partition numbers will need to be determined from the meta data using a customer written script.

EXEC DBMS_STATS.COPY_TABLE_STATS ('*SCHEMA*','SHORTWINDOW','*SYS_P800*','*SYS_P829*');

This approach assumes that yesterday's transactions are going to be similar to today's in terms of volume and relative cardinalities. In some cases this may not be true, so rather than copying statistics from today

it may be more appropriate to copy statistics from a number of days ago. However, this is only be done if data profiles are significant different and is causing performance problems.

6.2.2 Partitioned tables - statistics MSSQL

MSSQL has a number of automatic statistics gathering features:

- Auto Create Statistics - Default True
- Auto Update Statistics - Default True
- Auto Update Statistics Asynchronously - Default False

In general it is recommended, especially in high volume environments, that all of these are FALSE. During performance testing, with the default settings, spikes in CPU have been observed. However, turning these features off does mean more work for the DBA.

For most objects in the RT and NRT databases, statistics should be gathered after the cleanup process has successfully completed. However, if there is any significant load of data, such as new reference data or demographic data load, statistics should be gathered on the effected table. Another exception are the partitioned objects DETAIL and SHORTWINDOW statistics, statistics need to be gathered more often than daily on these tables.

When a new partition is created MSSQL assumes it is empty, and with all auto stats set to FALSE the optimizer will continue to assume it's empty until stats are gathered. This is an issue as for rule and UI performance statistics in Shortwindow and Detail must be up to date.

It is therefore recommended that incremental statistics are gathered on Shortwindow and Detail tables every 2 hours. This is a starting point and can be amended to suit the implementation, the optimal interval will depend on data profiles in the environment. For example if column cardinalities are consistent throughout the day, the optimizer may choose optimal access paths from statistics based on transactions from the first few hours of the day. However, when cardinalities are more random incremental statistics may need to be gathered every 30 minutes.

6.2.3 Partitioned tables – index rebuilds

Index rebuilds should be run for a partition after data for a new partition starts arriving in the table. By default for Shortwindow this will be after midnight and for DETAIL at the end of the week.

In most environments, after the partition cut over has occurred, there will be no or very few new rows inserted into historic partitions. Thus it should be possible to run online index rebuilds with little impact on transaction processing.

Starting with 9.0 (clean install), on MsSql, the Shortwindow, Detail and RT_Shortwindow tables are stored in one single filegroup with multiple files, to make it easy to increase or decrease the retention days.

However, in some environments a large percentage of transactions can arrive after the partition cut over, which will impact on when index rebuilds can safely be run. Thus it is important for the DBA to fully understand the type of environment in which they are working.

6.3 Enhanced profile tables

6.3.1 Enhanced profile tables - statistics

The enhanced profile tables can be considered volatile, in that data is being added or updated frequently all day and data is also deleted every day (depending on cycle length). It is therefore important that regular database maintenance is run against these tables. In addition new enhanced profiles required special consideration.

As a minimum, it is recommended that incremental statistics are gathered on a daily basis, however, they could also be gathered after every run of the calculation job.

When a new profile is defined data is added to the table during the next run of the enhanced profile job, no further action is required to activate the enhanced profile. When a new table is created it has no statistics, so the optimizer assumes it is empty. With each subsequent run of the calculation job data will be added to the new table, however, without any intervention the optimizer will still assume the table is empty. This may cause the optimizer to select a sub-optimal access path for the MERGE statement.

There are a number of potential options to address this issue:

Do nothing - If data volumes are low for the new profile it may not cause any issues

Temporarily enable Auto Stats for the new table - MSSQL only

Gather stats for the new table after every calculation job until data is representative of normal cardinalities

6.3.2 Enhanced profile tables - index rebuilds

The volatility and data profile of each enhanced profile will depend on the definition of the table, so it is not possible to suggest a solution which is appropriate for all enhanced profile tables. Optimal index rebuild frequency will have to be determined from:

- Analyzing statistics over time
- Cycle length (large amount of data deleted at end of cycle)
- Volume - It is not practical to rebuild large indexes every day
- Priority of rules which reference the Enhanced Profile table
- Maintenance window available.

However, as a starting point it is suggested index rebuilds are run at the end of each cycle.

7 Replication

If the NRT and RT schemas/databases reside on different database servers there may be a requirement to replicate some tables from NRT to RT. If NRT and RT reside on the same server RT rules can directly access NRT objects, however, due to network latency this is not practical if NRT and RT are on different servers.

7.1 Application replication

There are some tables which can optionally be replicated by the application. Section 7 of Application Administrator User Guide describes how to enable this replication. The tables replicated by feature are:

- MASTERDISP (RTR Action Disposition Update capability)
- CARD_31_DAYS_PEER_PROF (RTR Profile Data Access capability)
- CARD_31_DAYS_PROFILE (RTR Profile Data Access capability)
- PRM_REF_DATA (RTR Reference Data Access)

7.1.1 PRM_REF_DATA

Replication of this table is handled by the UI server. Data inserted by the UI into PRM_REF_DATA is also inserted into RT_PRM_REF_DATA.

However, if data is added or deleted from NRT via a non-PRM process then PP_RT_COPY_REF_DATA should be executed in order to synchronize the two tables.

7.1.2 MASTERDISP

There are INSERT/UPDATE/DELETE after triggers on the MASTERDISP table. These triggers log and changes to MASTERDISP_TEMP.

The UI server executes a number of stored procedures which uses the data in MASTERDISP_TEMP to synchronize RT_MASTERDISP with any changes in MASTERDISP. The frequency this process runs in configured during the setup of the capability.

7.1.3 CARD_31_DAYS_PEER_PROF & CARD_31_DAYS_PROFILE

These tables are populated by the batch process PP_CARD_PROFILE_JOB, which is typically run every day. If the capability is enabled, any new rows are stored in staging tables.

The UI server executes a number of stored procedures, which moves data from the staging tables in the NRT server to the RT counterpart tables. The data is moved via a database link. The frequency this process runs in configured during the setup of the capability.

For more information on how to enable this feature, please refer to the Application Administrator User Guide.

7.2 Enhanced profile tables

There is no capability within the PRM application to replicate enhanced profile tables from NRT to RT. However, it may be desirable replicate some of the enhanced profile tables to RT.

The Enterprise version of the RDBMS's have replication tools which can be used for this purpose. In addition there are 3rd party tools that could be used.

If an enhanced profile is being replicated, then there are restrictions on what changes can be made to that enhanced profile, including deleting it. If a native RDBMS replication technology is used the PRM application is able to identify enhanced profiles being replicated and will restrict UI functions available on that enhanced profile. However, if a third- party replication technology is being used the application is not able to determine if an enhanced profile is being replicated. In this case processes need to be put in place to ensure replicated enhanced profiles are not altered.

The restrictions are fully documented in section 15 of the *Application Administrator User Guide*.

The RT enhanced profile tables should only ever be read by RT processes, INSERT/UPDATE/DELETE operations will only come from NRT. This has an impact on the type of replication technology that can be used.

7.2.1 Creation steps

If an enhanced profile is to be replicated to RT, then replication should be set up immediately after is has been created, in order to minimize the amount of data transferred to RT during initial setup. When an enhanced profile is defined, the process that populates all enhanced profiles is amended to include the new enhanced profile. The next run of the enhanced profile job will start populating the new enhanced profile with data.

Therefore if an enhanced profiles is to be replicated; pausing the enhanced profile calculation job should be considered. Once replication is in place, the calculation job can be restarted. If this approach is adopted,

creation of an enhanced profile that is to be replicated should be carried out during a period of low activity, so the number of transactions to be processed by the calculation job does not become too high.

1. Disable enhanced profile job
2. Create new enhanced profile(s)
3. Configure replication so new table(s) are replicated
4. Enable enhanced profile job

7.2.2 Naming convention

All enhanced profile table names are prefixed with EP_.

A recommended best practice is to introduce a naming convention so it is easy to identify replicated enhanced profiles. This should be put in place if all enhanced profiles are to be replicated or only a subset.

If existing enhanced profiles become candidates for replication then they should be re-named before replication commences.

7.2.3 MSSQL

It is recommended that transactional replication is used which can be set up using SSMS.

The PRM application will disable any changes to any enhanced profile whose base table is being replicated by transactional replication. If this replication technology is not being used PRM will allow changes, even if they are being replicated by another technology.

7.2.4 Oracle

Oracle streams can be used to replicate tables, which can be set up in Enterprise Manager; however, this technology is deprecated in 12c. For 12c, Oracle recommends the use of Golden Gate or Materialized Query Views (MQTs). Golden gate is an additional cost option, if the institution already has a license for it, then it can be used. However, MQTs are a viable option which is included in the Enterprise Edition license.

The PRM application will disable any changes to any enhanced profile whose base table is being replicated by Oracle streams or MQTs. If either of these replication technologies are not being used PRM will allow changes, even if they are being replicated by another technology.

If MQTs are used, they need to be manually refreshed after the base tables in NRT have been updated. As MQTs are on a remote server on commit is not supported. It is therefore recommended that a second step be added to the enhanced profiles job which refreshes the RT enhanced profile tables. The sample script below could be used to achieve this:

```
BEGIN
 FOR REFRESH_MVIEWS IN (SELECT 'BEGIN DBMS_SNAPSHOT.REFRESH@<%PRMUER_RT%>(''' ||
PROFILE_TABLE|| '''); END;'AS  REFRESH_STATEMENT
   FROM USER_MVIEWS@<%PRMUSER_RT%> UM
    JOIN ENH_PROFILE_SETUP EPS ON UM.MVIEW_NAME = EPS.PROFILE_TABLE) LOOP
      DBMS_OUTPUT.PUT_LINE(REFRESH_MVIEWS.REFRESH_STATEMENT);
      EXECUTE IMMEDIATE REFRESH_MVIEWS.REFRESH_STATEMENT;
   END LOOP;

   END;
```

8 Database Security

8.1 Users

For the PRM application, there are 4 important database users. ACI does not stipulate the names of the users, any name which adheres to site standards is acceptable. The users are:

- NRT Install user - DB.NRT.dba.user
- RT Install user - DB.RT.dba.user
- NRT Application user - DB.NRT.user
- RT Application user - DB.RT.user

These users are defined in db.install.properties. In addition the application users will need to be defined in ag.install.properties and ui.install.properties.

8.2 Authority

The application users are created during installation if they don't already exist, all required authorities are granted during install time.

At installation time it is assumed the Install user has the following roles:

- MSSQL - sysadmin
- Oracle - CONNECT, DBA

The installation users are only used during installation, at other times they can be locked, or passwords changed with no impact on the application.

8.3 Passwords

As documented in the PRM Core Installation Guide, the passwords are stored in encrypted format in the installer properties files. Nevertheless, the access to these files and any backups, needs to be strictly controlled. It is especially recommended that the properties file and the cipher password file are not kept together – consider storing the cipher password file on a removable drive and backing it up on different storage medium.

Changing the PRM user password needs to be carefully considered as changing it will cause the application to fail. If the PRM user password does need to be changed the following process should be followed:

1. Stop AG, UI and any other processes connected using the PRM user
2. Change the password
3. Update database links with new password (Oracle only)
4. Re-install AG and UI with new password in config file
5. Start AG, UI and other processes

9 Performance

9.1 Adding new indexes

To improve the performance of a query adding a new index may seem an attractive option. Both MSSQL and Oracle may even suggest the definition of a new index that will support a slow running query.

However, new indexes should be created with caution. Adding new indexes will cause DML and database maintenance to consume more resources, in addition it may cause the optimizer to select the new index for queries it was not indented for.

Thus while adding the index that the RDBMS suggests may improve performance of an individual query, it may cause worse performance in the rest of the system.

Therefore, when a slow performing query is identified the questions should be asked before any new index is created, for example:

Why is this query being run?

Could it be run less often (e.g., additional entrance criteria for an aggregate/rule)

Could it be run at a different time of day (e.g., reports)

Can the SQL be changed to make it more efficient?

If it's a rule/aggregate have the rule best practices been followed?

Can the predicates be changed to make the query more restrictive (e.g., shorter period of time)

Could an enhanced profile be used?

The RDBMS is unable to answer these questions, which is why advice from the RDBMS should not be directly implemented.

9.1.1 Shortwindow

Despite the points made in the previous section about why new indexes should not be created, it is likely that new indexes will need to be added to Shortwindow. This is discussed in the rules best practice manual, however, there are a few points to consider when adding a new index to the Shortwindow table:

Could an existing index be extended rather than create a new index

Consider FILTER and INCLUDE options (MSSQL Only)

Consider function indexes, or indexes on virtual columns (Oracle Only)

Review existing indexes to ensure they are still appropriate for the rule set

9.1.2 Detail

Do not change any of the indexes on DETAIL, as these are required by the UI. In addition avoid adding new indexes to detail table.

If there is a rule which access the DETAIL table consider creating an enhanced profile rather than creating a new index to support the rule.

9.1.3 Other tables

Adding indexes to tables that are not in the transaction path will not cause transaction performance to degrade.

For example PRM_REF_DATA in some shops is referenced frequently by rules, so it may be beneficial to add multiple indexes to this table.

9.1.4 Review existing indexes

Rules, aggregates, Indexes, transactions, etc., change over time. Therefore an index that was created six months ago may no longer be as effective as it was when first created.

Every index will impact INSERT performance, so existing indexes on Shortwindow table should be regularly reviewed to ensure they continue to be beneficial.

9.2 Parameter peeking

Both MSSQL and Oracle optimizers can do parameter peeking; this is the optimizer looking at the actual values of parameters in SQL statements.

This functionality exists to maximize the amount of information the optimizer has about the query. This is fine for a single query; however, problems can occur if the query plan is cached and re-used by other queries whose parameters have significantly different cardinalities.

This issue can manifest itself by rule performance suddenly dropping. A work around is to re-deploy the rules which causes a re-compile however, in order to avoid this situation occurring the following approaches should be considered.

9.2.1 MSSQL

The optimizer hint OPTIMIZE FOR UNKNOWN prevents MSSQL from using the parameter peeking feature. This can be specified for the entire query, or individual predicates.

More information on how to use this optimizer hint can be found in the Microsoft documentation.

This forces the optimizer to calculate access path based on average statistics for the column rather than for a particular bind variable. This can mean sub optimal access path is chosen for some queries, however, it will ensure consistent access paths no matter what parameters are used.

9.2.2 Oracle

This feature is discussed in the Oracle documentation, and Oracle should be able to deal with different cardinalities of bind variables using adaptive cursor sharing. If Oracle determines different executions of the same query has very different statistics for bind variables it can generate different plans.

However, if Oracle does not detect bind variable correctly problems can occur. If this is found to be a problem the following session parameters can be set:

OPTIMIZER_INDEX_COST_ADJ = 10

CURSOR_SHARING = FORCE

_optim_peek_user_binds = false

This should cause Oracle us use indexes and stop bind variable peeking.

9.3 UI and DB NLS parameters (Oracle only)

The UI sessions should have the same NLS parameters as the database, especially the following parameters:

NLS_SORT

NLS_COMP

NLS_LANGUAGE

If these parameters do not match it can cause poor UI performance due to sorting data rather than using indexes to avoid the sort.

One approach to resolve this issue is to create a logon trigger for the PRM user which sets these session level parameters.

9.4 Performance troubleshooting

If there are performance issues, the first thing is to go through the DB maintenance checklist to ensure the database is being looked after correctly. If necessary re-read the relevant parts of this document.

If all of the steps have been followed then it is possible that:

There is a poor performing rule in the system

Long running reports are being executed

There is activity on the PRM database from an external system

Issue with the PRM software

In order to identify the source of the performance issue, the following questions should be answered:

When and how frequently does the problem happen

When was the issue first identified

Have there been any changes in the system recently:

New software applied

Index changes

Rule changes

New transaction types entering the system

Are PRM batch jobs scheduled correctly, including cleanup

Are DB maintenance jobs being run correctly

Are rules best practices being followed

What is the system behavior during periods of poor performance, e.g., high CPU, I/O

Is enough memory/SGA allocated to the database

Do any non PRM systems use the PRM database

What is the alert ratio

MSSQL (DM views and Activity Monitor) and Oracle (EM and AWR reports) both supply tools to help identify performance bottlenecks, including poor performing SQL.

9.4.1 Normal PRM behavior

In order to be able to identify poor performing SQL it is important to understand the normal behavior of the application. Within the transaction path the INSERT into detail is normally the most expensive operation followed by INSERT into SHORTWINDOW. If any rules are identified as more expensive than these then they should be investigated.

PRM issues a COMMIT after every transaction, thus the performance tools will report:

MSSQL - high logging waits

Oracle - high log file sync

This is relative to other waits in the system.

This is normal behavior for PRM, from a fraud detection perspective it is important that the database contains up to date committed data. If this does not happen velocity rules may not have all relevant transactions available thus not fire an alert when an alert should have been fired.

When a rule is deployed/redeployed, a new Rule procedure is created.

Only for SQL Server :

Persistant metadata for the entry procedure can become outdated because of change to their underlying objects and it is necessary to recompile it. Hence instead of using the existing plan for the same SP, the Database engine will create a new execution plan. Obtaining the compile exclusive lock and performing lookups and other work that is needed to reach this point can introduce a delay for the compile locks that leads to blocking. Be aware that this can introduce delays in stored procedure execution and cause unnecessarily high CPU utilization.

9.5 Data compression

On MSSQL, you can gain more space on database storage by enabling the COMPRESSION parameter (at PAGE level) by 3 times for big tables like: Detail, Shortwindow, enhance profiles, etc. Unfortunately, the queries will not be improved by having less blocks to load in memory, but also the extra CPU to

compress and decompress is insignificant. Of course, there are some generic queries which can estimate the compression rate on each table, before enabling this feature. The existing CLEANUP procedure for partitioned tables already support this feature.

10 DB Maintenance Checklist

10.1.1 Generic

Shortwindow index rebuilds - Completed daily for full partition

Detail index rebuilds - At a minimum run when new partition starts getting data

Non-partition cleanup run daily

Partition cleanup run daily

Relevant PRM batch jobs are scheduled

Rule best practices followed

10.2 Platform specific

10.2.1 Oracle

Shortwindow - Stats taken just before the end of day and copied to next partition

Oracle - Stats taken just before the interval and copied to next partition

Database and Session NLS parameters match

10.2.2 MSSQL

Shortwindow and Detail - Incremental Stats taken regularly; for example, every hour

11 Entry Procedures – Parameters That Control Behavior

Most of the parameters in the entry procedures contain transactional data; however, there are a few parameters which control the behavior of the entry procedure:

load_anyway

load_duplicate

mfupdate

selective_update

11.1 load_anyway

There are 2 possible values for this parameter:

0 – Generate an error if no matching portfolio is found

1- Continue processing even if no matching portfolio is found

This parameter controls behavior of the entry procedure no matching portfolio for the transaction can be found. This is for any of action levels assigned to the records source, if one or more matching portfolios are found processing continues unimpeded.

No rules will be run for this type of transaction, if load_anyway =1 then the transaction will be inserted into detail/shortwindow. In addition, the transaction will be applied to any relevant enhanced profiles.

This parameter is applicable in both NRT and RT.

11.2 Load_duplicate

There are 2 possible values for this parameter:

0 – Generate an error if there is an existing transaction in the system

1- Continue processing even if there is an existing transaction in the system

Within the transaction processing path there are a number of tables that are inserted into and have unique indexes. This parameter applies to all tables in the transaction path, including:

SHORTWINDOW

ALERTS

DETAIL_ALERTED_RULE

QueueHistory

Category_History

DETAIL

This parameter is only available in NRT, if there is duplicate in RT_SHORTWINOW it will fail.

11.3 Mf_update

This parameter only controls behavior in NRT, there are no updates in RT entry procedure. There are 2 possible values for this parameter:

0 - Update even if MASTER_TABLE.MF_DAT_TIM < DMFDateTime parameter

1 - Only update if MASTER_TABLE.MF_DAT_TIM < DMFDateTime parameter

However, an update will only ever be run if there one of the columns for that master table row has a different value to the incoming parameter. If they are all the same there is no need for the upgrade.

11.4 selective_update

This parameter only controls behavior in NRT, there are no updates in RT entry procedure. There are 2 possible values for this parameter:

0 – Update MASTER table column with input parameter value, whether it is NULL or NOT NULL

– Update MASTER table column with input parameter value if value is NOT NULL.

12 Enhanced Profile

12.1 Technical Overview

The table below provides an overview of the enhanced profile process.

01	Add a new Enhanced Profile	Config > Enhanced Profile Manager - Click on Add icon to add a new Enhanced Profile - Enter **Name**, Select **Shortwindow table**, **Cycle** (values: Daily, Weekly, Semi-Monthly, Monthly, Yearly, and Perpetual), **Cycle Date** (The transaction or demographic date field to use to determine the cycle to which the transaction belongs), **Number of cycles to retain, Primary profile** (The transaction or demographic string or the numeric primary field that will be profiled), **Sum field** (The transaction or demographic amount field to use to calculate the Sum, Min Amount, Max Amount, and Standard Deviation for the enhanced profile record), Choose **Summing negative amounts option**: Use negative amount or absolute amount, **Enter criteria** that must be met by a transaction in order for it to be included in the profile calculations. - Click on Save button to save changes Example **Name:** EP_CARD_ATM_COUNTRY **Description:** Profiles cards used at ATMs in countries outside of UK **Cycle:** Perpetual Num of cycles to retain: 1 **Cycle Date Field:** Transaction Date (DD_DATE) Primary Profile Field: SD_PAN Optional Profile Field: SD_TERM_CNTRY Sum Field: MD_TRAN_AMT1 **SQL:** SHORTWINDOW.SD_RETL_SIC_CDE = '6011' AND SHORTWINDOW.SD_TERM_CNTRY NOT IN ('GB','GBR')
02	Enhanced Profile Table Checks	When an Enhanced Profile is saved within the GUI, the stored procedure PP_ENH_PROFILE_SETUP is called, which performs the following operations: Calls PP_ENH_PROFILE_SYNTAX_CHK to validate the syntax within the Enhanced Profile. Inserts a new record into table ENH_PROFILE_SETUP. Check the table to ensure a record exists for your Enhanced Profile. Check that a table was created for your Enhanced Profile, so if you named your profile EP_CARD_ATM_COUNTRY, your table will be named EP_EP_CARD_ATM_COUNTRY The view PV_ENH_PROFILE_<ACTION_LEVEL_ID> will be updated depending on the primary profile fields used, to include your Enhanced Profile within the view SQL. Creates stored procedure PP_ENH_PROFILE_CALC
03	Load transactions into AG	The PP_NRT_ENTRY_* procedure calls PP_ENH_PROFILE_NEW_TXNS which writes the transactions into table ENH_PROFILE_NEW_TXNS_2 and the view PV_ENH_PROFILE_NEW_TXNS_WRITE selects transactions from this table.

04	Calculate Profile Data	Run PP_ENH_PROFILE_CALC. This stored procedure performs the following operations: Accesses ENH_PROFILE_SETUP for the setup details for each enhanced profile Selects transactions from PV_ENH_PROFILE_NEW_TXNS_WRITE and then inserts them into PV_ENH_PROFILE_NEW_TXNS_READ. Performs the enhanced profile calculations and then inserts the profile data into the relevant Enhanced Profile table e.g., EP_EP_CARD_ATM_COUNTRY Truncates view PV_ENH_PROFILE_NEW_TXNS_READ. Check that you have data in your Enhanced Profile tables.
05	Review Enhanced Profiles	Review > Enhanced Profiles Calls PV_ENH_PROFILE_DATA which selects the profile calculation data from each Enhanced Profile table and displays on the screen.
06	Schedule Jobs	If the customer intends to use Enhanced Profiling in the future, please schedule the jobs via Oracle Scheduler; - PP_ENH_PROFILE_CALC to run every 10 – 15 mins. - PP_ENH_PROFILE_CLEANUP to run every night.

12.2 Performance overview

Starting with PRM 8.8.0.0, ACI provides a partitioning feature for enhanced profile tables and enhanced profile calculation (temporary and staging) tables, in order to improve performance and facilitate the frequency group jobs usage.

By default, after 8800 upgrade, the lowest frequency group interval introduced has a one-hour time window-. This interval can be customized between a minimum value of 10 minutes and a maximum value of 3 hours (by default, the next frequency group). This interval (lowest) is used (and changed dynamically) by temporary (ENH_TEMP) and staging (ENH_PROFILE_NEW_TXNS) enhanced profile tables, to get the best performance during the data querying and replication.

After 8800 upgrade, the system will have created scheduled jobs for each frequency group (5 jobs), and one for cleanup. The existing job for enhanced profile calculation and for cleanup must be removed, in order to avoid the duplicate tasks. By default, the existing enhanced profiles will be assigned to the highest frequency group (24 hours), if they are not assigned to other frequency groups during the upgrade (db.config.properties file). The new cleanup job is configured to run every day.

The existing enhanced profile tables will remain the same (non-partitioned), and can be migrated to be partitioned by using additional scripts (see Core Installation Guide, 21 Appendix M).

The cleanup procedure automatically detects if the enhanced profile table is partitioned or not and applies the corresponding command: drop partition or delete rows.

The CHANGED column from ENH_PROFILE_FREQUENCY is used to set and get the frequency group status. To avoid the lock between UI and calculation jobs, each frequency group uses its own flag (stored in CHANGED column) to signal the current procedure (0 for no changes, 1 or 2 for changes and

procedure will to be recreated at the next run). Value 1 is set when an enhanced profile is moved between two frequency groups; value 2 is set when UI is changing the enhanced profile.

The LAST_FILL_ENH_TEMP_DATE column stores the last datetime value when the lowest frequency group was running and replicates the latest chunk of data joined from staging ENH_PROFILE_NEW_TXNS table with DETAIL table into temporary ENH_TEMP table. The same value is stored for all 5 frequency groups.

The LAST_CALCULATION_DATE column stores the last datetime value when the calculation job was done for each enhanced profile. So, when an enhanced profile is moved between two frequency groups, the calculation job will continue from this point and not from frequency group level.

The FREQUENCY_MOVE_DELAY_DATE column stores the delay time when an enhanced profile is moved between two frequency groups, and is the sum of both frequency groups' times, in order to avoid the possibility of running the calculation multiple times. This delay time should cover both cases when an enhanced profile is moved from lower frequency group to higher group, or from higher frequency group to lower group. At the next run of the new frequency group (where the enhanced profile was assigned) this column is checked for the assigned enhanced profile and will perform the calculation only when the delay is expired.

The ENH_TEMP table columns are built dynamically when a new enhanced profile is added, after checking its existence as primary or secondary field.

The new partitioned enhanced profile will have one unique index (with primary key constraint) based on PRIMARY_FIELD, SECONDARY_FIELD and CYCLE_END_DATE (partition column), and one index for secondary_field on first position only if it was defined.

12.2.1 **Mssql**

Enhanced profile temporary and staging tables have 72 partitions of 1 hour each (total 3 days) in advance.

12.2.1.1 Partitioning overview

The calculation job of the lowest frequency group will manage the partitions: by removing the aged partitions and creating them in advance. In case of failure, at the next run, the script automatically detects the aged partitions for removal and how many partitions need to be created, to keep the same number of partition days in advance.

The new enhanced profile tables will be created as partitioned tables, with the same number of partitions in the future as defined in the retention period, to avoid automatic row migration in case of the enhanced profile cleanup is accidentally stopped and started for the first time after many days. For example, for daily profile with 10 days retention, the enhanced profile table will have 10 (daily) partitions in the future. The cleanup procedure will manage and maintain the same number of partitions in the past and in the future.

These future partitions (created in advance) can have an impact on enhanced profile rules performance in case of missing CYCLE_END_DATE column filter condition or filtering after CYCLE_END_DATE column that includes the future dates. To avoid this performance degradation (between 10-20% of I/O and CPU), set the CYCLE_END_DATE filter until current day. For example, add the following condition “and CYCLE_END_DATE<=getdate()” to the enhanced profile rules.

For example, instead of the following enhanced profile rule:

EXISTS
(SELECT 1 FROM EP_CARD_ATM_DEBIT_ZIP (nolock)
WHERE PRIMARY_FIELD = @SD_PAN
AND SECONDARY_FIELD = @SD_TERM_POSTAL_CDE)

Or the next example:

EXISTS
(SELECT 1 FROM EP_CARD_ATM_DEBIT_ZIP (nolock)
WHERE PRIMARY_FIELD = @SD_PAN
AND SECONDARY_FIELD = @SD_TERM_POSTAL_CDE
AND CYCLE_END_DATE > @DD_DATE-5)

Add the CYCLE_END_DATE filter until current day condition:

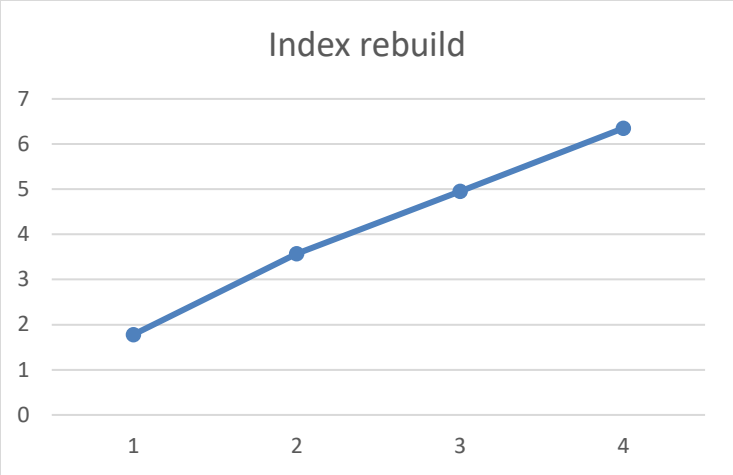
EXISTS
(SELECT 1 FROM EP_CARD_ATM_DEBIT_ZIP (nolock)
WHERE PRIMARY_FIELD = @SD_PAN
AND SECONDARY_FIELD = @SD_TERM_POSTAL_CDE
AND CYCLE_END_DATE > @DD_DATE-5
***AND CYCLE_END_DATE <= @DD_DATE**)*

Mssql has a 15000 partitions limitation. If the lowest frequency group is decreased from 1 hour to 10 minutes, the enhanced profile temporary and staging tables will have 144*3=432 partitions each. For example, if one calculation job is stopped (from any frequency group), the number of existing partitions can increase from 432 up to 15000 (after 100 days) thereby causing an error that the maximum number of partitions was exceeded.

The temporary and staging tables have a nonunique cluster index for performance reasons, because the temporary table is scanned for replication into staging, and the staging table is scanning to merge the rows into enhanced profile tables, in order to seek only on last partitions instead of performing full table scan.

12.2.1.2 Elapsed time estimation for migration of existing enhanced profile tables:

Step	**Elapsed time**			
Rows (million)	**28.5**	**57**	**85.5**	**114**
Create partition function and schema	1	1	1	1
Create partitioned index	106	213	296	380
Total (seconds)	107	214	297	381
Total (minutes)	**1.78**	**3.57**	**4.95**	**6.35**



MsSql supports adding/changing/removing enhanced profiles from the UI only when the frequency calculation job is not running at that time; otherwise, it will raise an error message: “Cannot perform the action. The job of frequency group 5 is currently running”. You need to wait until the calculation job ends, or you can disable it before any enhanced profile changes occur for that frequency group id.

12.2.2 **Oracle**

The temporary and staging tables don’t have an index, because when the temporary table is scanned for replication into staging, and the staging table is scanning to merge the rows into enhanced profile tables, the queries identify the right partitions instead of performing full table scan.

12.2.2.1 Parallel hint

The PARALLEL_ENABLED column stores the value to enable parallel calculation mode (for enhanced profile only) or not. By default, the value is set to 0 (false), and it can be enabled by setting to 1.

This parallel flag can be enabled only on Oracle, and it can be set for each frequency group. Usually, the lowest frequency group is a good candidate for parallelization. When the interval job is set to 10 minutes, it is not enough time to finish the staging table replications and the enhanced profiles assigned to the frequency group.

It is not enough to enable this flag, because you need to configure first the number of CPU cores to be used for parallel calculation (merge command).

The parallel configuration can be done by setting the following parameters from UI in SystemConfig dialog:

1. EnhProfileCalcHintMerge - string value (default null) for Enhanced Profile MERGE hint, for example: “parallel(t 5) “, where t is the alias of the EP table and 5 is the number of CPU cores to be used in parallel.

Any additional hints value can be added here.

2. EnhProfileCalcHintSelect - string value (default null) for Enhanced Profile SELECT hint, for example: “parallel(SHORTWINDOW 5) “, where SHORTWINDOW is the alias of the temporary ENH_TEMP table and 5 is the number of CPU cores to be used in parallel.

Any additional hints value can be added here.

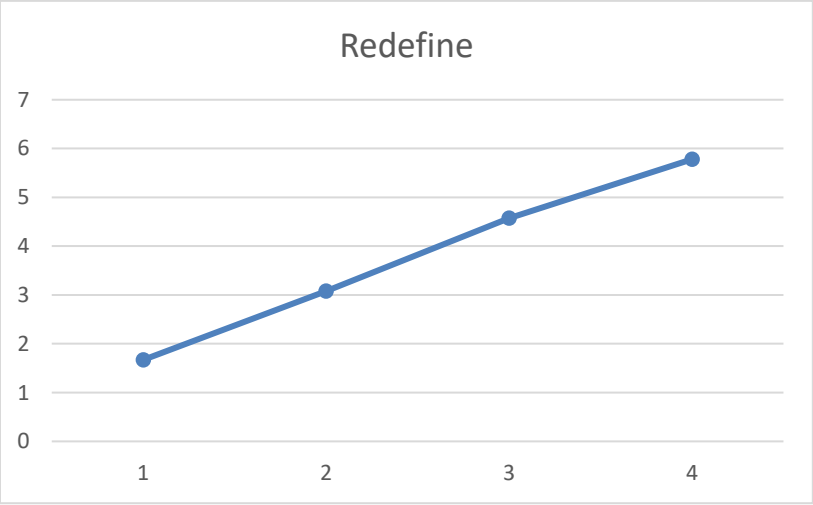
3. EnhProfileCalcHintLocalRAC – boolean value (default null), by setting to 1 which means true, and 0 or null means false.

This will enable the running of parallel commands only on local RAC node.

12.2.2.2 Elapsed time estimation for migration of existing enhanced profile tables

Redefine method:

Step	**Elapsed time**			
Rows (million)	**28.5**	**57**	**85.5**	**114**
Start redefine	34	70	101	131
Copy table dependents	41	71	111	140
Sync and finish redefine	1	1	1	1
Create partitioned index	24	43	61	75
Total (seconds)	100	185	274	347
Total (minutes)	**1.67**	**3.08**	**4.57**	**5.78**

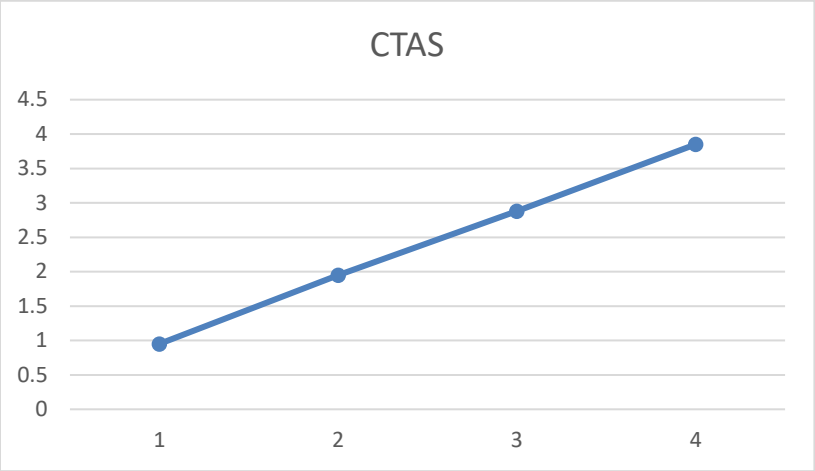


Note: Check if the PRM user has rights to execute DBMS_REDEFINITION package from sys.

CTAS (create table as) method:

Step	**Elapsed time**			
Rows (million)	**28.5**	**57**	**85.5**	**114**
CTAS	32	66	98	130

Create partitioned index	25	51	75	101
Total (seconds)	57	117	173	231
Total (minutes)	**0.95**	**1.95**	**2.88**	**3.85**



Oracle supports adding/changing/removing enhanced profiles from the UI when the frequency calculation job is running at that time.

13 Resource Allocation for Reports

13.1 Introduction

13.1.1 Purpose

The PRM UI needs the ability to run reports with lower priority, so that the overall system does not drastically drop in performance during reports execution. This can be turned on/off via configuration.

13.1.2 Intended audience

This document is aimed at anyone performing the PRM installation.

13.1.3 How to use

Use this document in conjunction with existing PRM installation and configuration documentation.

13.2 DB changes

PRM SYSTEMCONFIG table configuration

New configuration parameter :

1. CallStoredProcOnReportConns

- **True (1)** = connections from the reports data source will be treated by the resource governor with lower priority than the connections from the data source used by the rest of the PRM UI ;
- **false** (0)= connections from the reports data source will be treated by the resource governor with EQUAL priority as the connections from the data source used by the rest of the PRM UI

13.3 Oracle installation steps

13.3.1 Update SYSTEMCONFIG table

CallStoredProcOnReportConns parameter from SYSTEMCONFIG is set to **true** (1)

Log as PRM_USER and execute

```
UPDATE SYSTEMCONFIG
   SET LCFGVALUE = 1
   where SCFGVARIABLE= 'CallStoredProcOnReportConns';
COMMIT;
/
```

13.3.2 Configure Resource Manager

Resource allocation is managed by Resource Manager in Oracle Database.

Requirements:

- Resource plan : **PRM_PLAN** plan created and active (convention name recommended)
- Resource Groups: **PRM** and **PRMDBREPORTS** (convention name recommended)
- grant the switch privilege to user **PRM_CLIENT** (change PRM_CLIENT with user that is used to connect PRM to the tenant database). (mandatory)

A stored procedure is used to alter the session and switches to a different plan.

The Oracle DB Administrator can modify the stored procedure and change the plan name if he does not wish to use recommended convention name.

The stored procedure will be called by PRM Server only when a low priority wants to be used.

Configuration of Resource Manager is not in scope of this document. Customer's Database Administrator have to configure and set it accordingly with customer's system and needs.

13.4 Microsoft SQL Server installation steps

13.4.1 Update SYSTEMCONFIG table

For MSSQL, the DB Administrator needs to administer the resource plan and assign it to the new user through a User Defined Function that switches the work plan based on the user.

CallStoredProcOnReportConns parameter from SYSTEMCONFIG table is set to **true**,(1)

```
USE  <%PRM_DATABASE%>
GO
UPDATE SYSTEMCONFIG SET LCFGVALUE = 1 where SCFGVARIABLE= 'CallStoredProcOnReportConns';
GO
```

Change tokens with appropriate values.

13.4.2 Configure Reports user credential

In a Multitenant environment, the reports' user credentials are stored in root PRM_TENANTS table.

To configure these credentials or to update the password, it is necessary to execute following steps:

- You must be familiar with the installation/update/onboarding PRM process
- Install/upgrade PRM to multitenant environment
- Download and unzip files under REPORTS folder from additional_scripts.zip distribution file:
  additional_scripts.zip\Model_Upgrade\Upgrade\Additional_scripts\.<%DB%>\Reports
- Copy db.reports.<%DB%>.xml in ../install/ folder
- Copy db.reports.properties in ..//config/ folder
- Copy db.reports.<%DB%>.cmd (windows) or db.reports.ORACLE.sh (Unix) to ../shellscript folder
- Update in ../config/db.reports.properties file the following tokens:

  DB.NRT.root.url = URL connection for DB NRT ROOT

  DB.TENANT.organization = Tenant's organization

  DB.REPORTS.user= report user

DB.REPORTS.passwd = report user's password

- Execute from ..\shellscript folder db.reports.<%DB%>. batch file

or

from command line

ant -f ..\install\db.reports.ORACLE.xml reports

After execution of batch file, user and encrypted password for reports user are updated in ROOT PRM_TABLES and PRM_KEYS tables

13.4.3 Configure Resource Governor

Customer's Database Administrator have to configure if accordingly with existing system resources and customer's needs.

Requirements:

- **Resource Governor** enabled (mandatory)
- **PRM** and **PRMDBREPORTS** resource pool (recommended)
- **PRM** and **PRMDBREPORTS** Workload Groups (recommended)

PRM Resource Pool /workgroup is used for regular PRM user
PRMDBREPORTS Resource Pool /workgroup is used for regular PRM_report user

This convention name is a proposal, customer can have his own defined names

13.5 UI Server updates

13.5.1 Tomcat - Deprecated

See Appendix for more information regarding deprecated hardware and software platforms.

In order for the reports to run on a different resource plan on database, a new datasource needs to be provided in the UI server configuration file.

For Tomcat server installations, the PRMServer.xml should be updated as follows: a new datasource must be inserted, depending also on the database type:

- MSSQL
  For example, the basic datasource for NRT database looks like this:

```
<Resource name="PrmDB"
          auth="SERVLET"
          type="javax.sql.DataSource"
          scope="Shareable"
```

```
                description="Database for PRM Application"
                driverClassName="com.microsoft.sqlserver.jdbc.SQLServerDriver"
url="jdbc:sqlserver://localhost:1434;databasename=PRM84_NRT;SendStringParametersAsUni
code=false"
                username="prm_client"
                password="prma1ert"
                maxActive="10"
                maxIdle="10"
                maxWait="1000"
                testOnBorrow="true"
        validationQuery="SELECT 1"
        />
```

The new datasource will use a different resource name (***PrmDBReports*** - not customer's choice) and a different login, as described in 6.3. In our case, we defined it *prm_**client**_reports*:

```
        <Resource name="PrmDBReports"
                auth="SERVLET"
                type="javax.sql.DataSource"
                scope="Shareable"
                description="Database for PRM Application"
                driverClassName="com.microsoft.sqlserver.jdbc.SQLServerDriver"

url="jdbc:sqlserver://localhost:1434;databasename=PRM85_NRT;SendStringParametersAsUni
code=false"
                username="prm_client_reports"
                password="prma1ert"
                maxActive="10"
                maxIdle="10"
                maxWait="1000"
                testOnBorrow="true"
        validationQuery="SELECT 1"
          />
```

- Oracle
  The NRT datasource configuration for Oracle database looks like this:

```
          <Resource name="PrmDB"
```

```
        auth="SERVLET"
        type="javax.sql.DataSource"
        scope="Shareable"
        description="Database for PRM Application"
                driverClassName="oracle.jdbc.OracleDriver"
                url="jdbc:oracle:thin:@localhost:1521:xe"
        username="PRM85_NRT"
        password="prma1ert"
        maxActive="10"
        maxIdle="10"
        maxWait="1000"
        testOnBorrow="true"
/>
```

For the reports datasource, the only difference will be a different resource name (***PrmDBReports*** - not customer's choice); the username will stay the same.

```
<Resource name="PrmDBReports"
        auth="SERVLET"
        type="javax.sql.DataSource"
        scope="Shareable"
        description="Database for PRM Application"
                driverClassName="oracle.jdbc.OracleDriver"
                url="jdbc:oracle:thin:@localhost:1521:xe"
        username="PRM85_NRT"
        password="prma1ert"
        maxActive="10"
        maxIdle="10"
        maxWait="1000"
        testOnBorrow="true"
/>
```

13.5.2 WebSphere

Separate data source for reports on WebSphere Application Servers requires changes on XML configuration level and in the WAS console. XML changes occurs in two files that are distributed in PRMServer.ear and can be found inside this archive in PRMServer.war\WEB-INF\. The files are web.xml and ibm-web-bnd.xmi.

In web.xml, the new data source has to be added as context parameter and as resource reference, for example, as follows (as bolded text):

```
<web-app id="WebApp">
        <display-name>PRMServer</display-name>

        ……

        <context-param>
                <param-name>datasource</param-name>
                <param-value>java:comp/env/PrmDB</param-value>
        </context-param>

        <context-param>
                <param-name>datasource_reports</param-name>
                <param-value>java:comp/env/PrmDBReports</param-value>
        </context-param>

        <context-param>
                <param-name>datasource_realtime</param-name>
                <param-value>java:comp/env/PrmDBRealTime</param-value>
        </context-param>

        ……

        <resource-ref id="ResourceRef_1084563878927">
                <res-ref-name>PrmDB</res-ref-name>
                <res-type>java.lang.Object</res-type>
                <res-auth>Container</res-auth>
                <res-sharing-scope>Shareable</res-sharing-scope>
        </resource-ref>
        <resource-ref id="ResourceRef_1085415166326">
                <res-ref-name>PrmDBRealTime</res-ref-name>
                <res-type>java.lang.Object</res-type>
                <res-auth>Container</res-auth>
                <res-sharing-scope>Shareable</res-sharing-scope>
        </resource-ref>
```

```
        <resource-ref id="ResourceRef_1092085627783">
                <description>Mail Session</description>
                <res-ref-name>PrmMailSession</res-ref-name>
                <res-type>java.lang.Object</res-type>
                <res-auth>Container</res-auth>
                <res-sharing-scope>Shareable</res-sharing-scope>
        </resource-ref>

        <resource-ref id="ResourceRef_1084563878565">
                <res-ref-name>PrmDBReports</res-ref-name>
                <res-type>java.lang.Object</res-type>
                <res-auth>Container</res-auth>
                <res-sharing-scope>Shareable</res-sharing-scope>
        </resource-ref>

</web-app>
```

This newly created resource has to be binded to a JNDI name in ibm-web-bnd.xmi like this:

```
<webappbnd:WebAppBinding xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI"
xmlns:webappbnd="webappbnd.xmi" xmi:id="WebAppBinding_1107292076182"
virtualHostName="default_host">
  <webapp href="WEB-INF/web.xml#WebApp"/>
  <resRefBindings xmi:id="ResourceRefBinding_1212617816452" jndiName="PrmDB">
    <bindingResourceRef href="WEB-INF/web.xml#ResourceRef_1084563878927"/>
  </resRefBindings>
      ..............
  <resRefBindings xmi:id="ResourceRefBinding_1212617816455" jndiName="PrmDBReports">
    <bindingResourceRef href="WEB-INF/web.xml#ResourceRef_1084563878565"/>
  </resRefBindings>
</webappbnd:WebAppBinding>
```

There are also some changes that have to be brought on the WAS configuration in the WAS console.

- **MSSQL**
    - Create new J2C Authentication:

        - Go to **Security -> Global security** -> Java Authentication and Authorization Service **-> JAAS - J2C authentication data**
        - Press New
        - Insert an Alias you want (for example **PRM84_Reports**)
        - Insert **prm_client_reports** in User ID field
        - Insert the password for this login (**prma1ert**)
        - Enter a description
        - Click OK
        - Press Save to save into the master configuration
    - Create new datasource:
        - Go to **Resources -> JDBC -> Data sources**
        - Press New
        - Choose a Data source name and a JNDI name to use (for example you can use PrmDB_REPORTS for both). Click Next
        - In Select an existing JDBC provider section, you have to choose the same JDBC provider used by the NRT datasource. Click Next
        - In section Enter database specific properties for the data source, click Next
        - In section Setup security aliases, for Component-managed authentication alias and Container-managed authentication alias, select the **PRM84_Reports** alias newly created. Press Next
        - Press Finish
        - Press Save to save into the master configuration
    - Update PRMServer enterprise application
        - Fill the new PrmDbReports with the name you desire
        - Go to **Applications** -> Application Types **-> WebSphere enterprise applications**
        - Select the PRMServer application and click the Update button
        - In section Application update options, select Replace the entire application and choose your suitable way to replace the .ear file. Click Next
        - Choose Detailed installation type. Click Next
        - Click Next until you reach Step 7: Map resource references to resources. Here you can see that there is a new resource entry, with the Reference “PrmDBReports”. The Target Resource JNDI Name should be the JNDI name provided for the datasource newly created - PrmDB_REPORTS in our case. Click Next, and then, Continue.
        - Press Next for the rest of the steps
        - Press Finish at the last step (Step 12)
        - Press Save to save into the master configuration

- **ORACLE**
    - Create a new data source identical to the NRT one that is already defined – PrmDB, but change its name and the JNDI name
    - Update PRMServer enterprise application
        - Fill the new PrmDBReports with the name you desire
        - Go to **Applications** -> Application Types **-> WebSphere enterprise applications**
        - Select the PRMServer application and click the Update button

- In section Application update options, select Replace the entire application and choose your suitable way to replace the .ear file. Click Next
- Choose Detailed installation type. Click Next
- Click Next until you reach Step 7: Map resource references to resources. Here you can see that there is a new resource entry, with the Reference “PrmDBReports”. The Target Resource JNDI Name should be the JNDI name provided for the datasource newly created - PrmDB_REPORTS in our case. Click Next, and then, Continue.
- Press Next for the rest of the steps
- Press Finish at the last step (Step 12)

Press Save to save into the master configuration

14 SQL Server Always On Support

14.1 Background Information

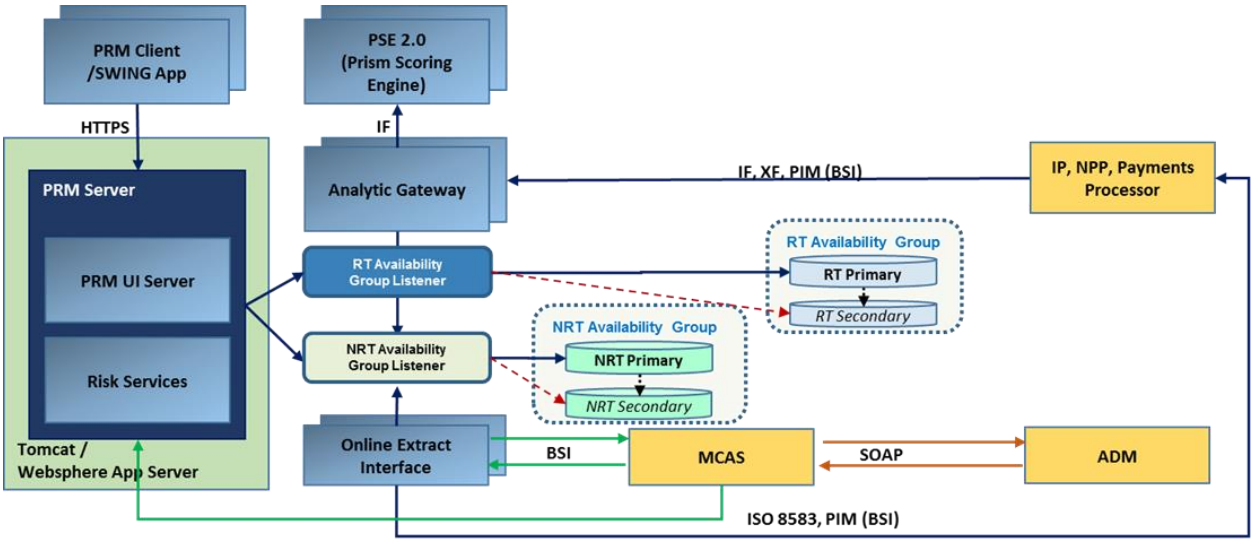
This section describes the SQL Server Always On configuration supported by ACI Proactive Risk Manager. It describes the expected behavior both during and after an Always On failover event.

ACI testing was completed using the Microsoft JDBC driver version 6.4, MS SQL Server 2014, and Tomcat 9. Analytic Gateway, the PRM user interface, and Online Extract Interface were configured and tested with both the asynchronous commit and synchronous commit failover modes. Please refer to the Microsoft Support Site that describes the failover and failover modes for additional information:

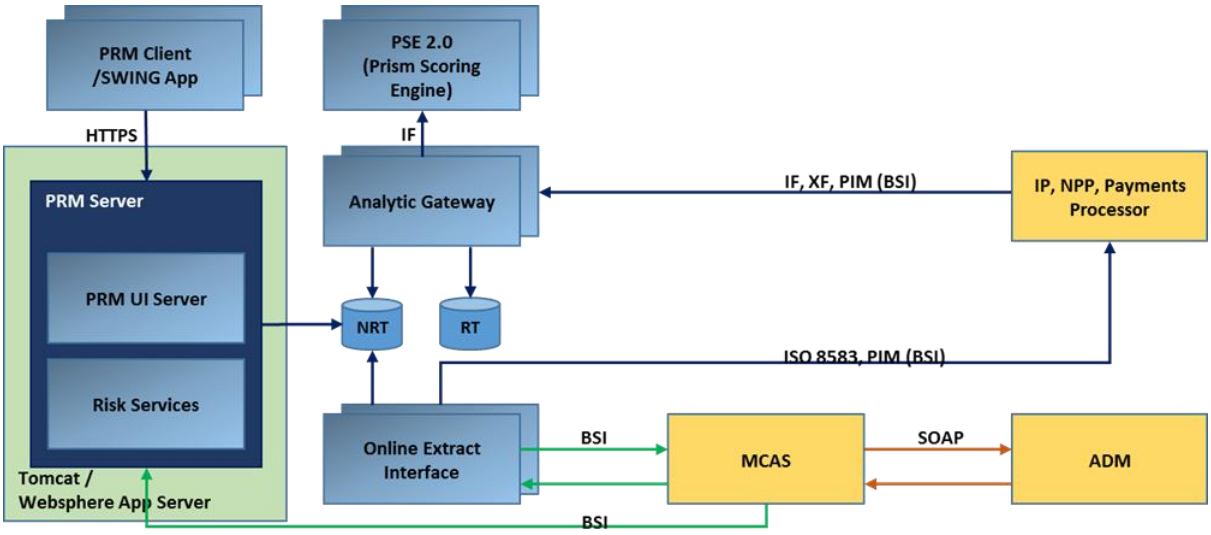
https://docs.microsoft.com/en-us/sql/database-engine/availability-groups/windows/failover-and-failover-modes-always-on-availability-groups

14.2 Configuration Overview

The following is an architectural diagram with Always On support via MSSQL clustering.

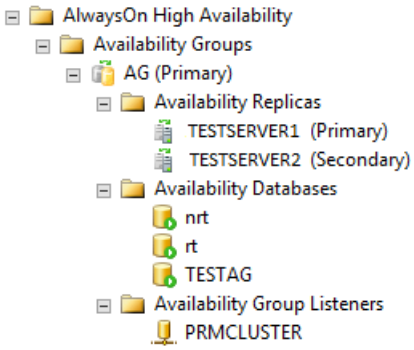


The following is an architectural block diagram of a standard deployment.



14.3 Configuration – SQL Server

To provide an example configuration, PRM was validated using Always On Availability Groups, with the following:

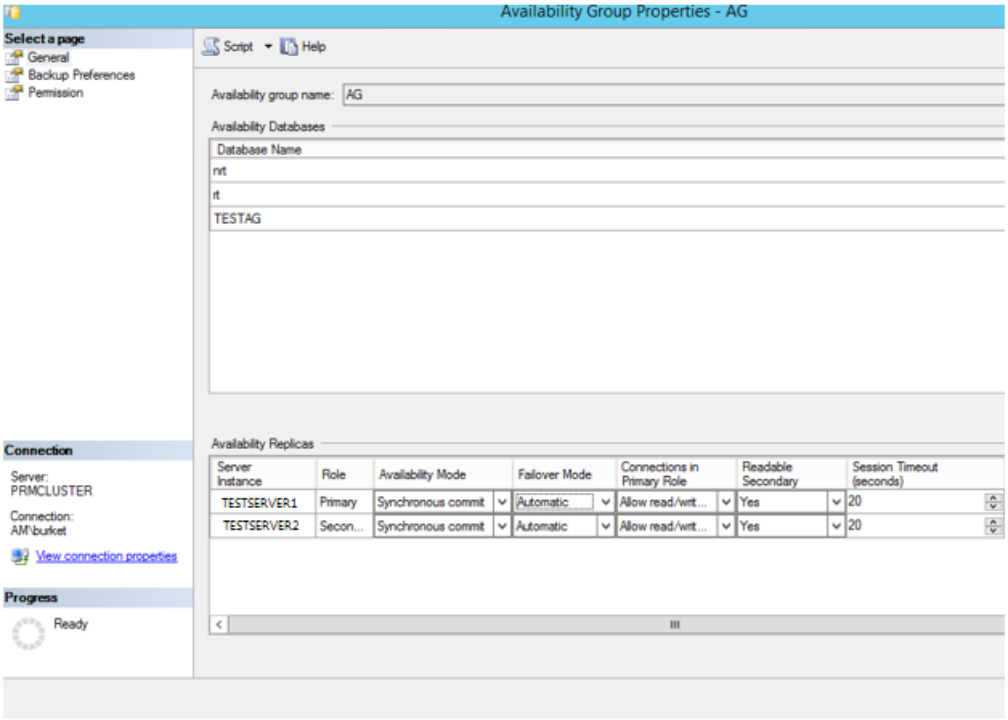


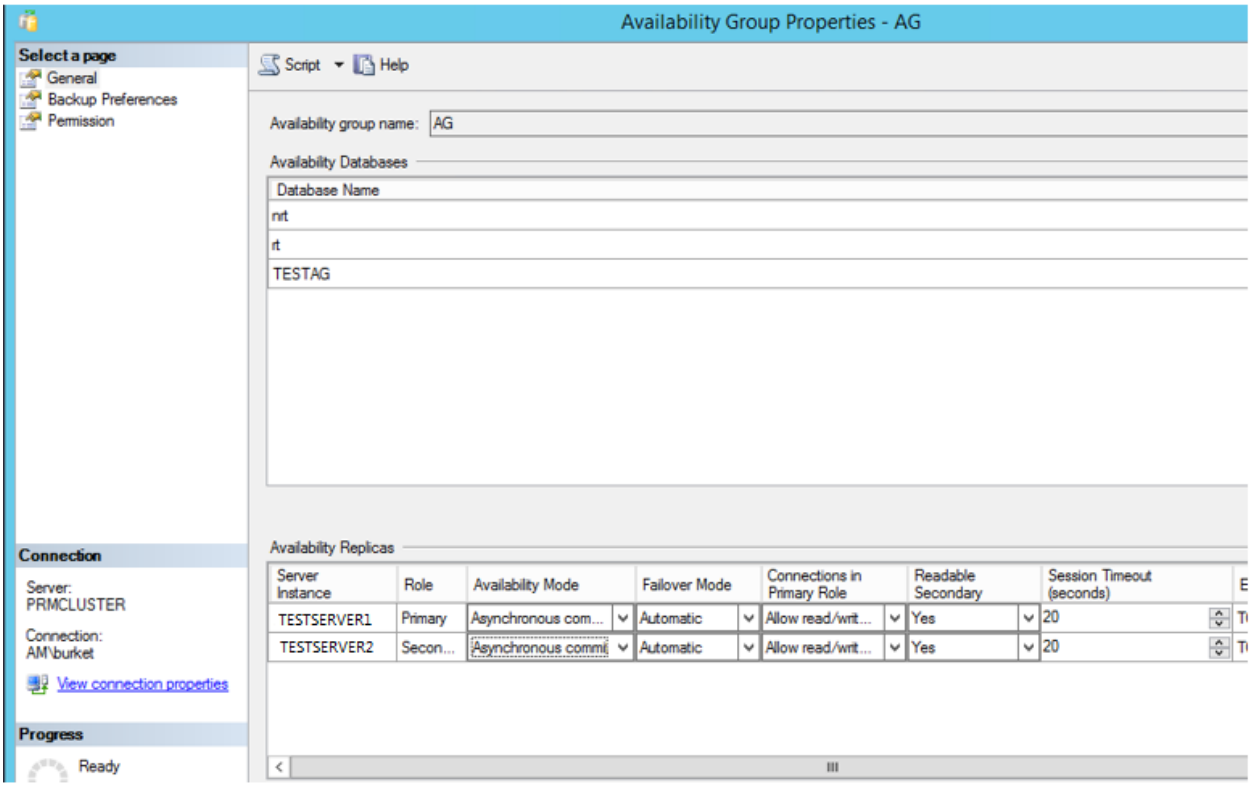
- TESTSERVER1 - Primary SQL Server TESTSERVER1
- TESTSERVER2 - Secondary SQL Server TESTSERVER2
- Databases – PRM NRT (Near Real-Time) and RT (Real-Time)
- PRMCLUSTER – Availability Group Listener

Information about Always On Availability Groups can be found at: https://msdn.microsoft.com/en-us/library/hh510230(v=sql.120).aspx

The Failover Cluster Manager configuration is outside the scope of this document, but the prerequisites described here must be met: https://msdn.microsoft.com/en-us/library/ff878487(v=sql.120).aspx#PrerequisitesForAGs

Availability Mode was tested with both Synchronous and Asynchronous commit.


Availability Group Properties - AG
Select a page
General
Backup Preferences
Permission
Script
Help
Availability group name:
AG
Availability Databases
Database Name
rnt
rt
TESTAG
Availability Replicas
Server Instance
Role
Availability Mode
Failover Mode
Connections in Primary Role
Readable Secondary
Session Timeout (seconds)
TESTSERVER1
Primary
Synchronous commit
Automatic
Allow read/writ...
Yes
20
TESTSERVER2
Secon...
Synchronous commit
Automatic
Allow read/writ...
Yes
20
Connection
Server:
PRMCLUSTER
Connection:
AM\burket
View connection properties
Progress
Ready


Availability Group Properties - AG
Select a page
General
Backup Preferences
Permission
Script
Help
Availability group name:
AG
Availability Databases
Database Name
rnt
rt
TESTAG
Availability Replicas
Server Instance
Role
Availability Mode
Failover Mode
Connections in Primary Role
Readable Secondary
Session Timeout (seconds)
TESTSERVER1
Primary
Asynchronous com...
Automatic
Allow read/writ...
Yes
20
TESTSERVER2
Secon...
Asynchronous commi
Automatic
Allow read/writ...
Yes
20
Connection
Server:
PRMCLUSTER
Connection:
AM\burket
View connection properties
Progress
Ready

14.4 Configuration - PRM Components

14.4.1 Analytic Gateway (AG) Configuration

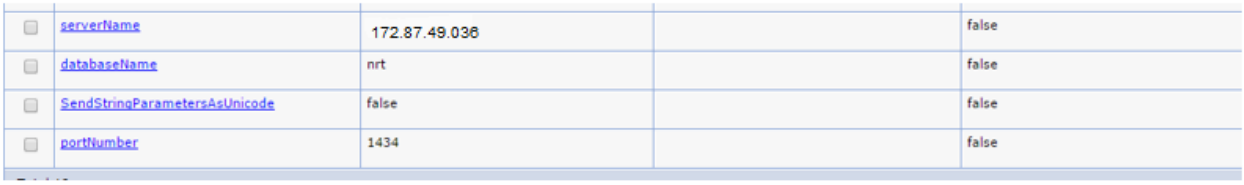
AG must be configured to point to the Availability Group Listener, not directly to the Primary server.

Config.AG<PROCESSID>

db.datasource=jdbc\:sqlserver\://PRMCLUSTER\:1434;databasename\=nrt;SendStringParametersAsUnicode\=false

14.4.2 Client Configuration

The PrmDB and PrmDbRealTime data sources must also be configured to point to the Availability Group Listener. Note in example PRMCLUSTER IP is 172.87.49.036. (Example is from WebSphere configuration)



☐	serverName	172.87.49.036		false
☐	databaseName	nrt		false
☐	SendStringParametersAsUnicode	false		false
☐	portNumber	1434		false

14.5 PRM Client Behavior on Failover

With the PRM system configured to use the Availability Group Listener instead of the SQL Server name of the primary or secondary replica, new connections will be directly routed to the current primary replica. Please see “Using a Listener to Connect to the Primary Replica” at the Availability Group Listeners, Client Connectivity, and Application Failover (SQL Server) site:

https://docs.microsoft.com/en-us/sql/database-engine/availability-groups/windows/listeners-client-connectivity-application-failover

Note, if you do not connect to the Listener, you will lose the benefit of new connections being directed automatically to the current primary replica. You will also lose the benefit of read-only routing. Also, note that if a client application connects directly to an instance of a SQL Server port and connects to a target database hosted in an availability group, and the target database is in primary state and online, then connectivity will succeed. If the target database is offline or in a transitional state, connectivity to the database will fail.

14.6 Behavior During Failover

When an availability group failover occurs, existing persistent connections to the availability group are terminated and the client must establish a new connection to continue working with the same primary database or read-only secondary database. While a failover is occurring on the server side, connectivity to the availability group may fail, forcing the client application to retry connecting until the primary is brought fully back online.

If the availability group comes back online during a client application’s connection attempt, but before the connect timeout period, the client driver may successfully connect during one of its internal retry attempts and no error will be surfaced to the application in this case.

An example from the AG trace and Event logs –

1. 2017-02-23 21:05:40,862 com.aciworldwide.framework.database.DBProcess.handleSqlException(DBProcess.java:4163) [Thread-11] - [14.2-Sp5.0 - Build 20160808-1620 revision 73642] SQLException occurred: java.sql.SQLException: I/O Error: Connection reset by peer: socket write error SQLState: 08S01 ErrorCode: 0
    a. Initial Error when failover starts, note above, “existing persistent connections to the availability group are terminated”. AG uses a connection pool to maintain persistent connections to DB for performance. These are terminated when failover occurs”
2. 2017-02-23 21:05:46,487 com.aciworldwide.framework.database.DBProcess.close(DBProcess.java:226) [Thread-11] - [14.2-Sp5.0 - Build 20160808-1620 revision 73642] Attempt to close DB connection when no connection exists.java.lang.Exception: Attempt to close an un-initialized connection.
    a. AG tries to close now invalid connection
3. 2017-02-23 21:05:46,402 com.aciworldwide.framework.database.DBConnectionManager$DBConnectionPool.newConnection(DBConnectionManager.java:1331) [Thread-11] - [AGNRT06] [3036] [11] [14.2-Sp5.0 - Build 20160808-1620 revision 73642] Unable to create a new SQL connection for jdbc:jtds:sqlserver://PRMCLUSTER:1434/nrt;SendStringParametersAsUnicode=false Exception: java.sql.SQLException: Network error IOException: Connection refused: connect
    a. 1st attempt to reconnect, note above “retry connecting until the primary is brought fully back online”

4. 2017-02-23 21:06:11,177 com.aciworldwide.framework.database.DBConnectionManager$DBConnectionPool.newConnection(DBConnectionManager.java:1331) [Thread-11] - [AGNRT06] [3036] [11] [14.2-Sp5.0 - Build 20160808-1620 revision 73642] Unable to create a new SQL connection for jdbc:jtds:sqlserver://PRMCLUSTER:1434/nrt;SendStringParametersAsUnicode=false Exception: java.sql.SQLException: Unable to access availability database 'nrt' because the database replica is not in the PRIMARY or SECONDARY role. Connections to an availability database is permitted only when the database replica is in the PRIMARY or SECONDARY role. Try the operation again later.
5. java.sql.SQLException: Unable to access availability database 'nrt' because the database replica is not in the PRIMARY or SECONDARY role. Connections to an availability database is permitted only when the database replica is in the PRIMARY or SECONDARY role. Try the operation again later.
    a. 2nd attempt to reconnect, note above “retry connecting until the primary is brought fully back online”
6. 2017-02-23 21:06:26,194 com.aciworldwide.framework.database.DBConnectionManager$DBConnectionPool.newConnection(DBConnectionManager.java:1322) [Thread-11] - [AGNRT06] [3036] [20] [14.2-Sp5.0 - Build 20160808-1620 revision 73642] Created New physical DB connection[108386171-3] (net.sourceforge.jtds.jdbc.JtdsConnection@675d77b) for jdbc:jtds:sqlserver://PRMCLUSTER:1434/nrt;SendStringParametersAsUnicode=false Catalog: nrt Isolation level: 2
    a. 3rd and successful reconnect, “primary is brought fully back online”

Note this is normal behavior based on how persistent (connection pooled) connections are handled at failover. AG will reconnect if it is configured to connect to Availability Listener and not directly to SQL Server.

From the Web application server (WebSphere):

1. [3/7/17 19:58:36:744 UTC] 0000009a WSJdbcPrepare W DSRA8600W: Error closing net.sourceforge.jtds.jdbcx.proxy.PreparedStatementProxy@c4671ccfjava.sql.SQLException: Connection has been returned to pool and this reference is no longer valid.
    a. Initial Error when failover starts, note above, “existing persistent connections to the availability group are terminated”. Websphere uses a connection pool to maintain persistent connections to DB for performance. These are terminated when failover occurs”
2. [3/7/17 19:58:36:754 UTC] 0000009a ConnectionEve W J2CA0206W: A connection error occurred. To help determine the problem, enable the Diagnose Connection Usage option on the Connection Factory or Data Source. This is the multithreaded access detection option. Alternatively check that the Database or MessageProvider is available.
3. [3/7/17 19:58:36:756 UTC] 0000009a ConnectionEve A J2CA0056I: The Connection Manager received a fatal connection error from the Resource Adapter for resource PrmDB. The exception is: java.sql.SQLException: I/O Error: Connection reset:java.net.SocketException: Connection reset
    a. Initial Error when failover starts, note above, “existing persistent connections to the availability group are terminated”. Websphere uses a connection pool to maintain persistent connections to DB for performance. These are terminated when failover occurs”
4. [3/7/17 19:58:59:307 UTC] 000000a3 SystemOut O 2017-03-07 19:58:59,307 ERROR [WebContainer : 8] 7OraMZjJep9WIWPeSI3zKyX 7OraMZjJep9WIWPeSI3zKyX - [PRM86] [7340] [8] The target database, 'nrt', is participating in an availability group and is currently not accessible for queries. Either data movement is suspended or the availability replica is not

enabled for read access. To allow read-only access to this and other databases in the availability group, enable read access to one or more secondary availability replicas in the group. For more information, see the ALTER AVAILABILITY GROUP statement in SQL Server Books Online. DSRA0010E: SQL State = S1000, Error Code = 976

a. Note above “retry connecting until the primary is brought fully back online”

5. [3/7/17 19:58:59:204 UTC] 000000a3 FfdcProvider W com.ibm.ws.ffdc.impl.FfdcProvider logIncident FFDC1003I: FFDC Incident emitted on /opt/IBM/WebSphere/AppServer/profiles/AppSrv01/logs/ffdc/server1_bccab39b_17.03.07_19.58.59.20359546383912510506 46.txt com.ibm.ws.rsadapter.spi.InternalGenericDataStoreHelper.getPooledCon 1298
6. [3/7/17 19:58:59:256 UTC] 000000a3 FfdcProvider W com.ibm.ws.ffdc.impl.FfdcProvider logIncident FFDC1003I: FFDC Incident emitted on /opt/IBM/WebSphere/AppServer/profiles/AppSrv01/logs/ffdc/server1_bccab39b_17.03.07_19.58.59.207484811107201488 5295.txt com.ibm.ejs.j2c.poolmanager.FreePool.createManagedConnectionWithMCWrapper 199
7. [3/7/17 19:58:59:274 UTC] 000000a3 FfdcProvider W com.ibm.ws.ffdc.impl.FfdcProvider logIncident FFDC1003I: FFDC Incident emitted on /opt/IBM/WebSphere/AppServer/profiles/AppSrv01/logs/ffdc/server1_bccab39b_17.03.07_19.58.59.271852594414804787 7011.txt com.ibm.ws.rsadapter.jdbc.WSJdbcDataSource.getConnection 299

a. Connection pool reconnects, “primary is brought fully back online”

In both these examples, the failover handling is working as designed. At a high level the flow is

1. Failover occurs, persistent connections are closed. This causes the “connection reset” error.
2. Existing connection is closed. Since it is now invalid, the error “Attempt to close an un-initialized connection.” Or similar error may occur.
3. Application attempts to reconnect. If the failover has not finished, the “refused connect”, or “target database is not available” errors may occur.
4. Once the failover is complete, application will continue with new connections.

14.7 Analytic Gateway Behavior

For AG, this means during the failover, transactions will not be processed and will be handled as if the database is down. AG will reconnect and continue processing once the failover is complete. It is normal to see a “Connection reset by peer” and/or “Network error IOException: Connection refused: connect” error when the connections are closed and application is waiting for failover to occur.

14.8 User Interface Behavior

14.8.1 WebSphere Application Server

The WebSphere connection pool will reset after the failover is complete. For the PRM Web application, any actions that are in process will return a connection error until the failover is complete. Once the failover is complete, the web application will continue. The user will not need to logout and login again, the current session will continue.

14.8.2 Apache Tomcat - Deprecated

See Appendix for more information regarding deprecated hardware and software platforms.

The Tomcat connection pool will reset, after the failover is complete. This is based on 4 configuration parameters in the JDBC JNDI configuration: (see Tomcat JDBC Pool for more information: https://tomcat.apache.org/tomcat-7.0-doc/jdbc-pool.html)

- factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
- validationQuery – valid SQL that will not throw an exception
- testOnBorrow – true
- validationInterval = # of milliseconds between testing connection ** Note by default this is set to 3600000 (one hour), which is too long for application to re-connect in timely manner. To support Always On the recommended interval is 3000-10000 (3 -10 seconds)

For the PRM Web application, any actions that are in process will return a connection error until the failover is complete. Once the failover is complete, the web application will continue. The user will not need to logout and login again, the current session will continue.

14.9 Online Extract Interface Behavior

The OEI process handles the failover the same as AG. They share the same AJI code that handles the database connection, so the behavior is the same. The application will log the connection reset, and any re-connect failures until the failover is complete, once that occurs, processing will continue as before.

14.10 Terminology

- **Synchronous-commit replicas** - support two settings—automatic or manual. The "automatic" setting supports both automatic failover and manual failover. To prevent data loss, automatic failover and planned failover require that the failover target be a synchronous-commit secondary replica with a healthy synchronization state (this indicates that every secondary database on the failover target is synchronized with its corresponding primary database). Whenever a secondary replica does not meet both conditions, it supports only forced failover. Note that forced failover is also supported for replicas whose role is in the RESOLVING state.
- **Asynchronous-commit replicas** - support only the manual failover mode. Moreover, because they are never synchronized, they support only forced failover.
- **Automatic failover** - A failover that occurs automatically on the loss of the primary replica. Automatic failover is supported only when the current primary and one secondary replica are both configured with failover mode set to AUTOMATIC and the secondary replica currently synchronized. If the failover mode of either the primary or secondary replica is MANUAL, automatic failover cannot occur.
- **Planned manual failover (without data loss)** - Planned manual failover, or manual failover, is a failover that is initiated by a database administrator, typically, for administrative purposes. A planned manual failover is supported only if both the primary replica and secondary replica are configured for synchronous-commit mode and the secondary replica is currently synchronized (in the SYNCHRONIZED state). When the target secondary replica is synchronized, manual failover (without data loss) is possible even if the primary replica has crashed because the secondary databases are ready for failover. A database administrator manually initiates a manual failover.
- **Forced failover (with possible data loss)** -A failover that can be initiated by a database administrator when no secondary replica is SYNCHRONIZED with the primary replica or the primary replica is not running and no secondary replica is failover ready. Forced failover risks possible data loss and is recommended strictly for disaster recovery. Forced failover is also known as forced manual failover because it can only be initiated manually. This is the only form of failover supported by in asynchronous-commit availability mode.
- **Automatic failover set** - Within a given availability group, a pair of availability replicas (including the current primary replica) that are configured for synchronous-commit mode with automatic

failover, if any. An automatic failover takes effect only if the secondary replica is currently SYNCHRONIZED with the primary replica.

- **Synchronous-commit failover set** - Within a given availability group, a set of two or three availability replicas (including the current primary replica) that are configured for synchronous-commit mode, if any. A synchronous-commit failover takes effect only if the secondary replicas are configured for manual failover mode and at least one secondary replica is currently SYNCHRONIZED with the primary replica.
- **Entire failover set** - Within a given availability group, the set of all availability replicas whose operational state is currently ONLINE, regardless of availability mode and of failover mode. The entire failover becomes relevant when no secondary replica is currently SYNCHRONIZED with the primary replica.

15 PRM Telemetry

15.1 Rules Event Alerting

The purpose of this functionality is to help people quickly see anomalies with generated alerts and is recommended to be used together with the Monitor Rule.

15.1.1 Description

A System or root admin, DBA or rule writer must configure and set this feature. Executing Rules Event Alerting feature, information about rules processed in an interval will be logged in database.

- User can define one or multiple polling frequencies for checking thresholds. ACI recommends a maximum of 10 to be active at the same time.
- Each entry in entry in configuration table can be activated or deactivated.
- For each entry the admin will be able to define the following:
  - event alerting rules - not to be confused with the PRM rules for capturing fraud.
  - actions in case an event is alerted
- The admin can define one or multiple event alerting rules for the same polling frequency.
- An event alerting rule will require the following information:
  - Event ID – Unique Id
  - Rule type for which the alerting applies: RT rules or NRT rules
  - Threshold - will be compared to the count of identified matches with the event alerting rule conditions in the polling interval
  - Optionally, an additional free SQL condition.
    - Example for RT rules: SD_REAL_TIME_DISPOSITION <> 'A'
    - Example for NRT rules: SD_TRAN_CDE_CAT IN (31, 32, 33, 39)
  - The action that can be assigned for a polling frequency is the following:
    - log in DB - details will be stored in the DB
  - Locale id for Rule's name to be used for logging
  - Debug Flag to be used for recording debug information in SYSTEMAUDIT table
  - The following information will be recorded when an event is alerted:
    - Rule ID
    - Rule name
    - Start date and time
    - End date and time
    - Count
    - Event ID of broken threshold
  - Information are obtained from NRT tables:
    - For NRT – DETAILALERTED_RULE and SHORTWINDOW
    - For RT -- SHORTWINDOW
    - For RT rules is mandatory to send information from RT flow to NRT flow (ND_REAL_TIME_RULE_FIRED field)
  - Alert date is:
    - For NRT – date and time of alert
    - For RT -- date and time that transaction processing started in database in NRT flow

Several performance recommendations should be considered with regards to polling intervals:

- NRT event alerting rules could be used for 30 second polling intervals or longer.
- RT event alerting rules could be used for 300 second polling intervals or longer, although performance may degrade on large Shortwindow tables. If performance is degraded, the feature should be disabled or an index be created on the ND_REAL_TIME_RULE_FIRED column on the Shortwindow table, considered only on frequent use of the feature.

Several performance recommendations should be considered with regards to indexing:

- For NRT when additional filtering (free SQL condition) is used, add indexes on (DD_PARTITION_DATE, ND_KEY_HASH, RECORD_SOURCE_ID, SD_TIEBREAKER, additional fields)
- For RT without additional filtering add index on (DD_APDATE, ND_REAL_TIME_RULE_FIRED)
- For RT with additional filtering add index on (DD_APDATE, ND_REAL_TIME_RULE_FIRED, additional fields)

Adding new indexes can have impact on overall performance issue. An analysis needs to be done if these indexes are needed and overall performance is impacted, and action needs to be taken accordingly.

15.1.2 Configuration DB table

Privileged user have to configure Event alerting for rules feature in RULES_OP_EVENTS_KPI table using a SQL interface. Following fields are available in table:

Field name	Type	Description
KPI_ID	Integer	Unique Id
RULE_TYPE	Character	Type of rule o be processed: NRT or RT
POLL_INTERVAL	Integer	Frequency (in seconds) for triggering KPI_ID event. Data will be collected for last POLL_INERVAL seconds before this entry is triggered
THRESHOLD	Integer	Value to be compared to the count of identified matches with the event alerting rule conditions in the polling interval. Only rules with count value exceeding threshold will be reported
NSTATUS	Integer	Status Flag 0 = Disabled 1 = enabled

DB_LOG	Integer	Flag to indicate if data will be logged in DB 0 = False 1= True
EMAIL_LOG	Integer	Reserved for future use
EMAIL_ADDRESS	National character	Reserved for future use
SSQL	National character	Additional free SQL condition. Example for RT rules: SHORTWINDOW.SD_REAL_TIME_DISPOSITION <> 'A' Example for NRT rules: SHORTWINDOW.SD_TRAN_CDE_CAT IN (31, 32, 33, 39) Remarks: Fully qualified names must be used to avoid collisions in names. Use syntax: TableName.ColumnName e.g. SHORTWINDOW.SD_TRAN_CDE_CAT
LOCALE_ID	Character	Locale Id used to display rule's name. If Rule's name in locale id is missing, will, be displayed name for 'en_US'. If second is missing too, will be displayed value from L10_PROPERTY table for Locale id = 'en_US' and property id = 0
BDEBUG	Integer	Flag to indicate if debug information will be recorded in SYSTEMAUDIT table. Information can be queried using filter ntype=11 and nsubtype=119 0 = False 1= True

By default, the following event alerting rules are available in the system in deactivated status:

- 30 sec polling frequency: event alerting rules for RT and NRT with threshold of 5; action: log in DB
- 60 sec polling frequency: event alerting rules for RT and NRT with threshold of 25; action: log in DB
- 300 sec polling frequency: event alerting rules for RT and NRT with threshold of 150; action: log in DB
- 600 sec polling frequency: event alerting rules for RT and NRT with threshold of 500; action: log in DB

– 1800 sec polling frequency: event alerting rules for RT and NRT with threshold of 2000; action: log in DB

15.1.3 One-time execution

Using a SQL interface, you can execute one-time rule's performance check for a defined event. Following code will check performance for event and, for rules with exceeded threshold, will log results in DB (if DB log flag is enabled) and return a dataset (if email log flag is enabled)

SQL Server

```
DECLARE  @log_id int
 DECLARE @RULES_OP_EVENTS_CALC TABLE
   (
    Rule_id INT
   , Rule_name NVARCHAR(500)
   , start_date DATETIME
   , end_date DATETIME
   , rule_count INT
   , kpi_id INT
   );
INSERT @RULES_OP_EVENTS_CALC
EXEC     PP_RULES_OP_EVENTS_CALC
         @KPI_ID = <%KPI ID%>

SELECT * FROM @RULES_OP_EVENTS_CALC
GO
```

Oracle

```
begin
PP_RULES_OP_EVENTS_CALC
    (  v_KPI_ID => <%KPI ID%>  );
end;
```

where <%KPI ID%> = KPI ID event to be computed

15.1.4 Automated execution scheduled using a pool interval

The execution of rule performance procedures is automated to run periodically based on a scheduled interval. An automated task is set to call the procedure recurrently based on two configuration values.

- The POLL_INTERVAL column from RULES_OP_EVENTS_KPI table represents the procedure trigger frequency (in seconds). An enabled event will run the procedure automatically each time based on the number of seconds specified in the interval value.
- To start the schedule of a event job, update NSTATU**S** column to value 1. To disable it update to value 0

E.g.:

```
UPDATE RULES_OP_EVENTS_KPI SET NSTATUS = 1 WHERE KPI_ID =<%KPI ID%>;
```

where <%KPI ID%> = KPI ID event to be computed

Any runtime configs changes for the RULES_OP_EVENTS_KPI table are checked each minute, so the changes are expected to be in process in maximum a minute. ACI recommends limiting the number of simultaneously enabled jobs to a maximum of 10.

15.1.5 DB Log structure

RULES_OP_EVENTS_LOG table – contain only records for rules with exceeded threshold

Field name	Type	Description
KPI_ID	Integer	Unique Id
RULE_ID	Integer	Rule ID
RULE_NAME	National character	Rule name for locale_id provided in configuration table
START_DATE	Timestamp	Start date
END_DATE	Timestamp	End date
KPI_ID	Integer	Event ID
RULE_COUNT	Integer	Number of alerts

15.1.6 DB Log cleanup

The nightly CLEANUP job calls PP_CLEANUPPARTITION procedure for partitioned tables or PP_CLEANUP_NON_PARTITIONED procedure for non-partitioned tables.

There is one non-partitioned table for the Rules Event Alerting Report defined in the CLEANUP_TABLES table.

The RULES_OP_EVENTS_LOG table contains log data for every active KPI defined to monitor alerted events on rules. The default retention period is 90 days.

The customer must configure cleanup interval for RULES_OP_EVENTS_LOG. Default provided values are

DAYSTOKEEP = 90,
DATECOLUMN= 'START_DATE'
BATCHSIZE = 100000
REPORTSIZE = 10
ADDWHERE = null
IS_PARTITIONED = 'N'

15.2 Rule Performance Monitoring

The Rule Performance Monitoring functionality is, as the name indicates, a way for you to monitor the performance of your NRT rules. If the functionality is enabled, information about the rules that run for a period of time that exceeds a configured threshold, expressed milliseconds, is logged to the RULEPERMONITOR_LOG table.

The Rule Performance Monitoring functionality is disabled by default. If you want to enable it contact your DBA, as this functionality can be enabled only through the database.
To enable it, the DBA must set the ThresholdMs column in the RULEPERFMONITOR_DEF table to a value greater than 0. ACI recommends starting with a value of 500 to 1000 (ms), watching the RULEPERFMONITOR_LOG table, and then modifying the setting as necessary.

The RULEPERFMONITOR_LOG table contains the following information for each rule that exceeds the configured threshold:
• ID of the rule
• Name of the rule
• Running time (expressed in milliseconds)
• Tiebreaker
• Date and time the action was logged (corresponds to the end time of the rule monitored)

Cleanup for the RULEPERFMONITOR_LOG table is performed by the CLEANUP_NON_PARTITIONED procedure, the same as for non-partitioned tables. There is an entry in the CLEANUP_TABLES table for the RULEPERFMONITOR_LOG table.

Cleanup is performed automatically after three days. It is performed using a batch size of 1000 (1000 rows at a time) to avoid lock escalation and contention between the Rules procedure inserting rows in the RULEPERFMONITOR_LOG table and the CLEANUP_NON_PARTITIONED procedure, if run at the same time, deleting rows.

16 Appendix

Deprecated Hardware and Software Platforms

ACI strives to provide support for hardware and software platforms, as driven by customer demand and industry standards. As part of our ongoing product development processes, we continually re-evaluate these platforms to ensure we strike a balance between meeting market demand and being able to provide high quality releases in a timely manner. As a result, we periodically need to discontinue support for certain hardware and or software platforms.

As of the PRM 9.0 release, the following changes are being made:

- AIX will no longer be a supported server operating system. RedHat Linux can be used as an alternative.
- Support for the Apache Tomcat application server has been deprecated. It can be used with the Desktop client for 9.0, but it is no longer being actively tested in our releases and will be removed in PRM release 9.1 once all remaining user interface features have been ported to the Web UI.